

Claude Code Learning Handbook

Portrait English Edition

Claude Code Learning Handbook

Portrait English edition for screen reading.

This edition is generated from the same handbook structure as the Chinese edition, with all visible text rendered in English to avoid missing-glyph squares in PDF readers.

Table of Contents

Part 1: Getting Oriented

- Chapter 1: Why Claude Code Is More Than a Chatbot
- Chapter 2: Environment, Account, Models, and Your First Task
- Chapter 3: Files, Edits, Shell Commands, and Tools
- Chapter 4: Preparing a Learning Project

Part 2: Daily Development Workflows

- Chapter 5: Code Understanding and Project Maps
- Chapter 6: Bug Localization and Fixing
- Chapter 7: Refactoring, Testing, and Documentation
- Chapter 8: Git Workflow and Commit Quality
- Chapter 9: From One-off Questions to Reusable Workflows

Part 3: Markdown, Memory, and Knowledge Bases

- Chapter 10: Markdown as an Agent Control Layer
- Chapter 11: The CLAUDE.md Memory System
- Chapter 12: Project Rules, AGENTS.md, and Rule Folders
- Chapter 13: Local Knowledge Bases
- Chapter 14: Memory System Anti-patterns

Part 4: Prompting and Model Control

- Chapter 15: Claude Code Prompting Fundamentals
- Chapter 16: Plan Mode and Exploration Mode
- Chapter 17: Taming Opus 4.7: Models, Effort, and Thinking Budget
- Chapter 18: Prompt Orchestration for Complex Engineering Tasks

Part 5: Context, Tokens, and Cost

- Chapter 19: Context Windows and File Selection
- Chapter 20: Compaction, Recovery, and Long-session Management
- Chapter 21: Token Saving and Cost Optimization

Part 6: Extension Systems

- Chapter 22: Skills Fundamentals

- Chapter 23: Custom Commands and Task-oriented Skills
- Chapter 24: SubAgents Fundamentals
- Chapter 25: Multi-Agent Collaboration
- Chapter 26: Hooks for Event-driven Automation
- Chapter 27: MCP Fundamentals and External Tool Integration
- Chapter 28: The Plugins System
- Chapter 29: Tools and Rules
- Chapter 30: Combination Patterns for Skill, Agent, MCP, and Plugin
- Chapter 31: Mini Project: Building an Extension System

Part 7: Automation and CI

- Chapter 32: Headless Mode and Command-line Automation
- Chapter 33: Agent SDK Introduction
- Chapter 34: Routines, Scheduled Tasks, and Automated Workflows
- Chapter 35: GitHub PR and CI Workflows
- Chapter 36: IDE, Browser, and External Tool Integration

Part 8: Security and Risk Control

- Chapter 37: Permissions and Safe Working Modes
- Chapter 38: Sandboxes, Data Security, and Prompt Injection
- Chapter 39: Git, Safe Releases, and Rollbacks
- Chapter 40: Production-grade Claude Code Usage Guidelines

Part 9: Advanced Harness Engineering

- Chapter 41: Claude Code Internal Mechanics
- Chapter 42: Context Engineering, Caching, and Cost Engineering
- Chapter 43: From Personal Capability to Team AI Coding Systems

Part 10: Complete Case Studies

- Chapter 44: Case Study 1: From Bug to PR in a Python Project
- Chapter 45: Case Study 2: Team Claude Code Toolkit

Chapter 1: Why Claude Code Is More Than a Chatbot

Part 1: Getting Oriented
Main case language: Python
Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Why Claude Code Is More Than a Chatbot into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Why Claude Code Is More Than a Chatbot is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a Python example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small Python project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-01"
claude "Work only on: Why Claude Code Is More Than a Chatbot. Explain the plan, then
show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-01
claude "Work only on: Why Claude Code Is More Than a Chatbot. Explain the plan, then
show the verification checklist."

from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Contents

- 1.1 Concepts and positioning
 - 1.1.1 Definition and use case
 - 1.1.2 Practical procedure
 - 1.1.3 Verification and boundary
- 1.2 Workflow and operating model

- 1.2.1 Definition and use case
- 1.2.2 Practical procedure
- 1.2.3 Verification and boundary
- 1.3 Common mistakes and risk control
- 1.3.1 Definition and use case
- 1.3.2 Practical procedure
- 1.3.3 Verification and boundary
- 1.4 Practice, deliverables, and review
- 1.4.1 Definition and use case
- 1.4.2 Practical procedure
- 1.4.3 Verification and boundary

1.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 1.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 1.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 1.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-01"
claude "Work only on: Why Claude Code Is More Than a Chatbot: Concepts and
positioning. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-01
claude "Work only on: Why Claude Code Is More Than a Chatbot: Concepts and
positioning. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

1.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 1.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 1.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 1.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-01"
```

```
claude "Work only on: Why Claude Code Is More Than a Chatbot: Workflow and operating model. Explain the plan, then show the verification checklist."
```

```
# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-01
claude "Work only on: Why Claude Code Is More Than a Chatbot: Workflow and operating model. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

1.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 1.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 1.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 1.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.

3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-01"
claude "Work only on: Why Claude Code Is More Than a Chatbot: Common mistakes and risk
control. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-01
claude "Work only on: Why Claude Code Is More Than a Chatbot: Common mistakes and risk
control. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

1.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 1.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 1.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 1.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-01"
claude "Work only on: Why Claude Code Is More Than a Chatbot: Practice, deliverables,
and review. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-01
claude "Work only on: Why Claude Code Is More Than a Chatbot: Practice, deliverables,
and review. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed Why Claude Code Is More Than a Chatbot as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Environment, Account, Models, and Your First Task.

Chapter 2: Environment, Account, Models, and Your First Task

Part 1: Getting Oriented

Main case language: C#

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Environment, Account, Models, and Your First Task into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Environment, Account, Models, and Your First Task is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a C# example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small C# project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-02"
claude "Work only on: Environment, Account, Models, and Your First Task. Explain the
plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-02
claude "Work only on: Environment, Account, Models, and Your First Task. Explain the
plan, then show the verification checklist."

using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Contents

- 2.1 Concepts and positioning
- 2.1.1 Definition and use case

- 2.1.2 Practical procedure
- 2.1.3 Verification and boundary
- 2.2 Workflow and operating model
- 2.2.1 Definition and use case
- 2.2.2 Practical procedure
- 2.2.3 Verification and boundary
- 2.3 Common mistakes and risk control
- 2.3.1 Definition and use case
- 2.3.2 Practical procedure
- 2.3.3 Verification and boundary
- 2.4 Practice, deliverables, and review
- 2.4.1 Definition and use case
- 2.4.2 Practical procedure
- 2.4.3 Verification and boundary

2.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 2.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 2.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 2.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-02"
claude "Work only on: Environment, Account, Models, and Your First Task: Concepts and
```

```
positioning. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-02
claude "Work only on: Environment, Account, Models, and Your First Task: Concepts and
positioning. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

2.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 2.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 2.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 2.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-02"
claude "Work only on: Environment, Account, Models, and Your First Task: Workflow and
operating model. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-02
claude "Work only on: Environment, Account, Models, and Your First Task: Workflow and
operating model. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

2.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an

engineering behavior rather than a label.

Important subtopics:

- 2.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 2.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 2.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-02"
claude "Work only on: Environment, Account, Models, and Your First Task: Common
mistakes and risk control. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-02
claude "Work only on: Environment, Account, Models, and Your First Task: Common
mistakes and risk control. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

2.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 2.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 2.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 2.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-02"
claude "Work only on: Environment, Account, Models, and Your First Task: Practice,
deliverables, and review. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-02
claude "Work only on: Environment, Account, Models, and Your First Task: Practice,
deliverables, and review. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed Environment, Account, Models, and Your First Task as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Files, Edits, Shell Commands, and Tools.

Chapter 3: Files, Edits, Shell Commands, and Tools

Part 1: Getting Oriented
Main case language: C++
Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Files, Edits, Shell Commands, and Tools into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Files, Edits, Shell Commands, and Tools is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a C++ example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small C++ project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-03"
claude "Work only on: Files, Edits, Shell Commands, and Tools. Explain the plan, then
show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-03
claude "Work only on: Files, Edits, Shell Commands, and Tools. Explain the plan, then
show the verification checklist."

#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Contents

- 3.1 Concepts and positioning
 - 3.1.1 Definition and use case
 - 3.1.2 Practical procedure
 - 3.1.3 Verification and boundary

- 3.2 Workflow and operating model
 - 3.2.1 Definition and use case
 - 3.2.2 Practical procedure
 - 3.2.3 Verification and boundary
- 3.3 Common mistakes and risk control
 - 3.3.1 Definition and use case
 - 3.3.2 Practical procedure
 - 3.3.3 Verification and boundary
- 3.4 Practice, deliverables, and review
 - 3.4.1 Definition and use case
 - 3.4.2 Practical procedure
 - 3.4.3 Verification and boundary

3.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 3.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 3.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 3.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code   \book\examples\chapter-03"
claude "Work only on: Files, Edits, Shell Commands, and Tools: Concepts and
positioning. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-03
claude "Work only on: Files, Edits, Shell Commands, and Tools: Concepts and
```

positioning. Explain the plan, then show the verification checklist."

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

3.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 3.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 3.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 3.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.

5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-03"
claude "Work only on: Files, Edits, Shell Commands, and Tools: Workflow and operating
model. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-03
claude "Work only on: Files, Edits, Shell Commands, and Tools: Workflow and operating
model. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

3.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 3.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 3.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 3.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-03"
claude "Work only on: Files, Edits, Shell Commands, and Tools: Common mistakes and
risk control. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-03
claude "Work only on: Files, Edits, Shell Commands, and Tools: Common mistakes and
risk control. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

3.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 3.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 3.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 3.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-03"
claude "Work only on: Files, Edits, Shell Commands, and Tools: Practice, deliverables,
and review. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-03
claude "Work only on: Files, Edits, Shell Commands, and Tools: Practice, deliverables,
and review. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format

Problem	Symptom	Fix
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed Files, Edits, Shell Commands, and Tools as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Preparing a Learning Project.

Chapter 4: Preparing a Learning Project

Part 1: Getting Oriented

Main case language: Python

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Preparing a Learning Project into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Preparing a Learning Project is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a Python example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small Python project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-04"
claude "Work only on: Preparing a Learning Project. Explain the plan, then show the
verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-04
claude "Work only on: Preparing a Learning Project. Explain the plan, then show the
verification checklist."

from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Contents

- 4.1 Concepts and positioning
 - 4.1.1 Definition and use case
 - 4.1.2 Practical procedure
 - 4.1.3 Verification and boundary
- 4.2 Workflow and operating model
 - 4.2.1 Definition and use case
 - 4.2.2 Practical procedure

- 4.2.3 Verification and boundary
- 4.3 Common mistakes and risk control
- 4.3.1 Definition and use case
- 4.3.2 Practical procedure
- 4.3.3 Verification and boundary
- 4.4 Practice, deliverables, and review
- 4.4.1 Definition and use case
- 4.4.2 Practical procedure
- 4.4.3 Verification and boundary

4.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 4.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 4.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 4.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-04"
claude "Work only on: Preparing a Learning Project: Concepts and positioning. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-04
claude "Work only on: Preparing a Learning Project: Concepts and positioning. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()
```

```
def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

4.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 4.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 4.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 4.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-04"
claude "Work only on: Preparing a Learning Project: Workflow and operating model.
Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-04
claude "Work only on: Preparing a Learning Project: Workflow and operating model."
```

Explain the plan, then show the verification checklist."

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

4.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 4.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 4.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 4.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.


```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-04"
claude "Work only on: Preparing a Learning Project: Common mistakes and risk control.
Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-04
claude "Work only on: Preparing a Learning Project: Common mistakes and risk control.
Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

4.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 4.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 4.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 4.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.

2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-04"
claude "Work only on: Preparing a Learning Project: Practice, deliverables, and
review. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-04
claude "Work only on: Preparing a Learning Project: Practice, deliverables, and
review. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.

- A small local case that can be repeated.

Chapter Summary

This chapter framed Preparing a Learning Project as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Code Understanding and Project Maps.

Chapter 5: Code Understanding and Project Maps

Part 2: Daily Development Workflows

Main case language: C#

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Code Understanding and Project Maps into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Code Understanding and Project Maps is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a C# example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small C# project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-05"
claude "Work only on: Code Understanding and Project Maps. Explain the plan, then show
the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-05
claude "Work only on: Code Understanding and Project Maps. Explain the plan, then show
the verification checklist."

using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Contents

- 5.1 Concepts and positioning
- 5.1.1 Definition and use case

- 5.1.2 Practical procedure
- 5.1.3 Verification and boundary
- 5.2 Workflow and operating model
- 5.2.1 Definition and use case
- 5.2.2 Practical procedure
- 5.2.3 Verification and boundary
- 5.3 Common mistakes and risk control
- 5.3.1 Definition and use case
- 5.3.2 Practical procedure
- 5.3.3 Verification and boundary
- 5.4 Practice, deliverables, and review
- 5.4.1 Definition and use case
- 5.4.2 Practical procedure
- 5.4.3 Verification and boundary

5.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 5.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 5.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 5.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-05"
claude "Work only on: Code Understanding and Project Maps: Concepts and positioning."
```

Explain the plan, then show the verification checklist."

```
# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-05
claude "Work only on: Code Understanding and Project Maps: Concepts and positioning.
Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

5.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 5.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 5.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 5.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-05"
claude "Work only on: Code Understanding and Project Maps: Workflow and operating
model. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-05
claude "Work only on: Code Understanding and Project Maps: Workflow and operating
model. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

5.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an

engineering behavior rather than a label.

Important subtopics:

- 5.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 5.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 5.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-05"
claude "Work only on: Code Understanding and Project Maps: Common mistakes and risk control. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-05
claude "Work only on: Code Understanding and Project Maps: Common mistakes and risk control. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

5.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 5.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 5.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 5.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-05"
claude "Work only on: Code Understanding and Project Maps: Practice, deliverables, and
review. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-05
claude "Work only on: Code Understanding and Project Maps: Practice, deliverables, and
review. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed Code Understanding and Project Maps as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Bug Localization and Fixing.

Chapter 6: Bug Localization and Fixing

Part 2: Daily Development Workflows

Main case language: C++

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Bug Localization and Fixing into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Bug Localization and Fixing is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a C++ example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small C++ project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-06"
claude "Work only on: Bug Localization and Fixing. Explain the plan, then show the
verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-06
claude "Work only on: Bug Localization and Fixing. Explain the plan, then show the
verification checklist."

#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Contents

- 6.1 Concepts and positioning
 - 6.1.1 Definition and use case
 - 6.1.2 Practical procedure
 - 6.1.3 Verification and boundary
- 6.2 Workflow and operating model
 - 6.2.1 Definition and use case

- 6.2.2 Practical procedure
- 6.2.3 Verification and boundary
- 6.3 Common mistakes and risk control
- 6.3.1 Definition and use case
- 6.3.2 Practical procedure
- 6.3.3 Verification and boundary
- 6.4 Practice, deliverables, and review
- 6.4.1 Definition and use case
- 6.4.2 Practical procedure
- 6.4.3 Verification and boundary

6.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 6.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 6.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 6.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-06"
claude "Work only on: Bug Localization and Fixing: Concepts and positioning. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-06
claude "Work only on: Bug Localization and Fixing: Concepts and positioning. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

6.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 6.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 6.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 6.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-06"
```

```
claude "Work only on: Bug Localization and Fixing: Workflow and operating model.  
Explain the plan, then show the verification checklist."
```

```
# macOS / Linux  
cd ~/claude-code-handbook/book/examples/chapter-06  
claude "Work only on: Bug Localization and Fixing: Workflow and operating model.  
Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>  
#include <iostream>  
  
int main() {  
    std::filesystem::path target{"README.md"};  
    std::cout << "Path: " << std::filesystem::absolute(target) << "  
";  
    std::cout << "Exists: " << std::filesystem::exists(target) << "  
";  
    return 0;  
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

6.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 6.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 6.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 6.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.

2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-06"
claude "Work only on: Bug Localization and Fixing: Common mistakes and risk control.
Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-06
claude "Work only on: Bug Localization and Fixing: Common mistakes and risk control.
Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

6.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 6.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.

- 6.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 6.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-06"
claude "Work only on: Bug Localization and Fixing: Practice, deliverables, and review.
Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-06
claude "Work only on: Bug Localization and Fixing: Practice, deliverables, and review.
Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context

Problem	Symptom	Fix
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed Bug Localization and Fixing as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Refactoring, Testing, and Documentation.

Chapter 7: Refactoring, Testing, and Documentation

Part 2: Daily Development Workflows

Main case language: Python

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Refactoring, Testing, and Documentation into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Refactoring, Testing, and Documentation is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a Python example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small Python project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-07"
claude "Work only on: Refactoring, Testing, and Documentation. Explain the plan, then
show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-07
claude "Work only on: Refactoring, Testing, and Documentation. Explain the plan, then
show the verification checklist."

from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Contents

- 7.1 Concepts and positioning
 - 7.1.1 Definition and use case
 - 7.1.2 Practical procedure
 - 7.1.3 Verification and boundary
- 7.2 Workflow and operating model

- 7.2.1 Definition and use case
- 7.2.2 Practical procedure
- 7.2.3 Verification and boundary
- 7.3 Common mistakes and risk control
- 7.3.1 Definition and use case
- 7.3.2 Practical procedure
- 7.3.3 Verification and boundary
- 7.4 Practice, deliverables, and review
- 7.4.1 Definition and use case
- 7.4.2 Practical procedure
- 7.4.3 Verification and boundary

7.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 7.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 7.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 7.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-07"
claude "Work only on: Refactoring, Testing, and Documentation: Concepts and
positioning. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-07
claude "Work only on: Refactoring, Testing, and Documentation: Concepts and
positioning. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

7.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 7.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 7.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 7.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-07"
```

```
claude "Work only on: Refactoring, Testing, and Documentation: Workflow and operating model. Explain the plan, then show the verification checklist."
```

```
# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-07
claude "Work only on: Refactoring, Testing, and Documentation: Workflow and operating model. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

7.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 7.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 7.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 7.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.

3. Require file paths, commands, logs, or citations as evidence.

4. Ask for a plan before any risky action.

5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-07"
claude "Work only on: Refactoring, Testing, and Documentation: Common mistakes and
risk control. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-07
claude "Work only on: Refactoring, Testing, and Documentation: Common mistakes and
risk control. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

7.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 7.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 7.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 7.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-07"
claude "Work only on: Refactoring, Testing, and Documentation: Practice, deliverables,
and review. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-07
claude "Work only on: Refactoring, Testing, and Documentation: Practice, deliverables,
and review. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed Refactoring, Testing, and Documentation as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Git Workflow and Commit Quality.

Chapter 8: Git Workflow and Commit Quality

Part 2: Daily Development Workflows

Main case language: C#

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Git Workflow and Commit Quality into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Git Workflow and Commit Quality is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a C# example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small C# project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-08"
claude "Work only on: Git Workflow and Commit Quality. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-08
claude "Work only on: Git Workflow and Commit Quality. Explain the plan, then show the verification checklist."

using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) || Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Contents

- 8.1 Concepts and positioning
 - 8.1.1 Definition and use case
 - 8.1.2 Practical procedure
 - 8.1.3 Verification and boundary

- 8.2 Workflow and operating model
 - 8.2.1 Definition and use case
 - 8.2.2 Practical procedure
 - 8.2.3 Verification and boundary
- 8.3 Common mistakes and risk control
 - 8.3.1 Definition and use case
 - 8.3.2 Practical procedure
 - 8.3.3 Verification and boundary
- 8.4 Practice, deliverables, and review
 - 8.4.1 Definition and use case
 - 8.4.2 Practical procedure
 - 8.4.3 Verification and boundary

8.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 8.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 8.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 8.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-08"
claude "Work only on: Git Workflow and Commit Quality: Concepts and positioning.
Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-08
claude "Work only on: Git Workflow and Commit Quality: Concepts and positioning."
```

Explain the plan, then show the verification checklist."

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

8.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 8.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 8.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 8.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.

3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-08"
claude "Work only on: Git Workflow and Commit Quality: Workflow and operating model.
Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-08
claude "Work only on: Git Workflow and Commit Quality: Workflow and operating model.
Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

8.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 8.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 8.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 8.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-08"
claude "Work only on: Git Workflow and Commit Quality: Common mistakes and risk control. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-08
claude "Work only on: Git Workflow and Commit Quality: Common mistakes and risk control. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) || Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

8.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.

- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 8.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 8.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 8.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-08"
claude "Work only on: Git Workflow and Commit Quality: Practice, deliverables, and
review. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-08
claude "Work only on: Git Workflow and Commit Quality: Practice, deliverables, and
review. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.

- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed Git Workflow and Commit Quality as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with From One-off Questions to Reusable Workflows.

Chapter 9: From One-off Questions to Reusable Workflows

Part 2: Daily Development Workflows

Main case language: C++

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of From One-off Questions to Reusable Workflows into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what From One-off Questions to Reusable Workflows is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a C++ example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small C++ project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-09"
claude "Work only on: From One-off Questions to Reusable Workflows. Explain the plan,
then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-09
claude "Work only on: From One-off Questions to Reusable Workflows. Explain the plan,
then show the verification checklist."

#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Contents

- 9.1 Concepts and positioning
 - 9.1.1 Definition and use case
 - 9.1.2 Practical procedure
 - 9.1.3 Verification and boundary

- 9.2 Workflow and operating model
 - 9.2.1 Definition and use case
 - 9.2.2 Practical procedure
 - 9.2.3 Verification and boundary
- 9.3 Common mistakes and risk control
 - 9.3.1 Definition and use case
 - 9.3.2 Practical procedure
 - 9.3.3 Verification and boundary
- 9.4 Practice, deliverables, and review
 - 9.4.1 Definition and use case
 - 9.4.2 Practical procedure
 - 9.4.3 Verification and boundary

9.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 9.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 9.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 9.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-09"
claude "Work only on: From One-off Questions to Reusable Workflows: Concepts and
positioning. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-09
claude "Work only on: From One-off Questions to Reusable Workflows: Concepts and
```

positioning. Explain the plan, then show the verification checklist."

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

9.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 9.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 9.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 9.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.

5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-09"
claude "Work only on: From One-off Questions to Reusable Workflows: Workflow and
operating model. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-09
claude "Work only on: From One-off Questions to Reusable Workflows: Workflow and
operating model. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

9.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 9.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 9.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 9.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-09"
claude "Work only on: From One-off Questions to Reusable Workflows: Common mistakes
and risk control. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-09
claude "Work only on: From One-off Questions to Reusable Workflows: Common mistakes
and risk control. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

9.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 9.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 9.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 9.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-09"
claude "Work only on: From One-off Questions to Reusable Workflows: Practice,
deliverables, and review. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-09
claude "Work only on: From One-off Questions to Reusable Workflows: Practice,
deliverables, and review. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format

Problem	Symptom	Fix
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed From One-off Questions to Reusable Workflows as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Markdown as an Agent Control Layer.

Chapter 10: Markdown as an Agent Control Layer

Part 3: Markdown, Memory, and Knowledge Bases

Main case language: Python

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Markdown as an Agent Control Layer into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Markdown as an Agent Control Layer is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a Python example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small Python project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-10"
claude "Work only on: Markdown as an Agent Control Layer. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-10
claude "Work only on: Markdown as an Agent Control Layer. Explain the plan, then show the verification checklist."

from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Contents

- 10.1 Concepts and positioning
 - 10.1.1 Definition and use case
 - 10.1.2 Practical procedure
 - 10.1.3 Verification and boundary
- 10.2 Workflow and operating model

- 10.2.1 Definition and use case
- 10.2.2 Practical procedure
- 10.2.3 Verification and boundary
- 10.3 Common mistakes and risk control
- 10.3.1 Definition and use case
- 10.3.2 Practical procedure
- 10.3.3 Verification and boundary
- 10.4 Practice, deliverables, and review
- 10.4.1 Definition and use case
- 10.4.2 Practical procedure
- 10.4.3 Verification and boundary

10.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 10.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 10.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 10.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-10"
claude "Work only on: Markdown as an Agent Control Layer: Concepts and positioning.
Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-10
claude "Work only on: Markdown as an Agent Control Layer: Concepts and positioning.
Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

10.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 10.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 10.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 10.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-10"
```

```
claude "Work only on: Markdown as an Agent Control Layer: Workflow and operating model. Explain the plan, then show the verification checklist."
```

```
# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-10
claude "Work only on: Markdown as an Agent Control Layer: Workflow and operating model. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

10.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 10.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 10.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 10.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.

3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-10"
claude "Work only on: Markdown as an Agent Control Layer: Common mistakes and risk
control. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-10
claude "Work only on: Markdown as an Agent Control Layer: Common mistakes and risk
control. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

10.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 10.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 10.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 10.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-10"
claude "Work only on: Markdown as an Agent Control Layer: Practice, deliverables, and
review. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-10
claude "Work only on: Markdown as an Agent Control Layer: Practice, deliverables, and
review. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed Markdown as an Agent Control Layer as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with The CLAUDE.md Memory System.

Chapter 11: The CLAUDE.md Memory System

Part 3: Markdown, Memory, and Knowledge Bases

Main case language: C#

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of The CLAUDE.md Memory System into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what The CLAUDE.md Memory System is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a C# example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small C# project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-11"
claude "Work only on: The CLAUDE.md Memory System. Explain the plan, then show the
verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-11
claude "Work only on: The CLAUDE.md Memory System. Explain the plan, then show the
verification checklist."

using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Contents

- 11.1 Concepts and positioning
 - 11.1.1 Definition and use case
 - 11.1.2 Practical procedure
 - 11.1.3 Verification and boundary

- 11.2 Workflow and operating model
 - 11.2.1 Definition and use case
 - 11.2.2 Practical procedure
 - 11.2.3 Verification and boundary
- 11.3 Common mistakes and risk control
 - 11.3.1 Definition and use case
 - 11.3.2 Practical procedure
 - 11.3.3 Verification and boundary
- 11.4 Practice, deliverables, and review
 - 11.4.1 Definition and use case
 - 11.4.2 Practical procedure
 - 11.4.3 Verification and boundary

11.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 11.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 11.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 11.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-11"
claude "Work only on: The CLAUDE.md Memory System: Concepts and positioning. Explain
the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-11
claude "Work only on: The CLAUDE.md Memory System: Concepts and positioning. Explain
```

the plan, then show the verification checklist."

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

11.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 11.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 11.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 11.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.

3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-11"
claude "Work only on: The CLAUDE.md Memory System: Workflow and operating model.
Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-11
claude "Work only on: The CLAUDE.md Memory System: Workflow and operating model.
Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

11.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 11.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 11.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 11.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-11"
claude "Work only on: The CLAUDE.md Memory System: Common mistakes and risk control.
Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-11
claude "Work only on: The CLAUDE.md Memory System: Common mistakes and risk control.
Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

11.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.

- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 11.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 11.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 11.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-11"
claude "Work only on: The CLAUDE.md Memory System: Practice, deliverables, and review.
Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-11
claude "Work only on: The CLAUDE.md Memory System: Practice, deliverables, and review.
Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.

- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed The CLAUDE.md Memory System as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Project Rules, AGENTS.md, and Rule Folders.

Chapter 12: Project Rules, AGENTS.md, and Rule Folders

Part 3: Markdown, Memory, and Knowledge Bases

Main case language: C++

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Project Rules, AGENTS.md, and Rule Folders into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Project Rules, AGENTS.md, and Rule Folders is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a C++ example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small C++ project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-12"
claude "Work only on: Project Rules, AGENTS.md, and Rule Folders. Explain the plan,
then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-12
claude "Work only on: Project Rules, AGENTS.md, and Rule Folders. Explain the plan,
then show the verification checklist."

#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Contents

- 12.1 Concepts and positioning
 - 12.1.1 Definition and use case
 - 12.1.2 Practical procedure
 - 12.1.3 Verification and boundary

- 12.2 Workflow and operating model
- 12.2.1 Definition and use case
- 12.2.2 Practical procedure
- 12.2.3 Verification and boundary
- 12.3 Common mistakes and risk control
- 12.3.1 Definition and use case
- 12.3.2 Practical procedure
- 12.3.3 Verification and boundary
- 12.4 Practice, deliverables, and review
- 12.4.1 Definition and use case
- 12.4.2 Practical procedure
- 12.4.3 Verification and boundary

12.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 12.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 12.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 12.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-12"
claude "Work only on: Project Rules, AGENTS.md, and Rule Folders: Concepts and
positioning. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-12
claude "Work only on: Project Rules, AGENTS.md, and Rule Folders: Concepts and
```

positioning. Explain the plan, then show the verification checklist."

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

12.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 12.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 12.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 12.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.

5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-12"
claude "Work only on: Project Rules, AGENTS.md, and Rule Folders: Workflow and
operating model. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-12
claude "Work only on: Project Rules, AGENTS.md, and Rule Folders: Workflow and
operating model. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

12.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 12.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 12.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 12.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-12"
claude "Work only on: Project Rules, AGENTS.md, and Rule Folders: Common mistakes and
risk control. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-12
claude "Work only on: Project Rules, AGENTS.md, and Rule Folders: Common mistakes and
risk control. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

12.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 12.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 12.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 12.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-12"
claude "Work only on: Project Rules, AGENTS.md, and Rule Folders: Practice,
deliverables, and review. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-12
claude "Work only on: Project Rules, AGENTS.md, and Rule Folders: Practice,
deliverables, and review. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format

Problem	Symptom	Fix
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed Project Rules, AGENTS.md, and Rule Folders as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Local Knowledge Bases.

Chapter 13: Local Knowledge Bases

Part 3: Markdown, Memory, and Knowledge Bases

Main case language: Python

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Local Knowledge Bases into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Local Knowledge Bases is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a Python example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small Python project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-13"
claude "Work only on: Local Knowledge Bases. Explain the plan, then show the
verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-13
claude "Work only on: Local Knowledge Bases. Explain the plan, then show the
verification checklist."

from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Contents

- 13.1 Concepts and positioning
 - 13.1.1 Definition and use case
 - 13.1.2 Practical procedure
 - 13.1.3 Verification and boundary
- 13.2 Workflow and operating model
 - 13.2.1 Definition and use case
 - 13.2.2 Practical procedure

- 13.2.3 Verification and boundary
- 13.3 Common mistakes and risk control
- 13.3.1 Definition and use case
- 13.3.2 Practical procedure
- 13.3.3 Verification and boundary
- 13.4 Practice, deliverables, and review
- 13.4.1 Definition and use case
- 13.4.2 Practical procedure
- 13.4.3 Verification and boundary

13.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 13.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 13.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 13.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-13"
claude "Work only on: Local Knowledge Bases: Concepts and positioning. Explain the
plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-13
claude "Work only on: Local Knowledge Bases: Concepts and positioning. Explain the
plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()
```

```
def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

13.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 13.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 13.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 13.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-13"
claude "Work only on: Local Knowledge Bases: Workflow and operating model. Explain the
plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-13
claude "Work only on: Local Knowledge Bases: Workflow and operating model. Explain the
```

```
plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

13.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 13.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 13.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 13.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.


```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-13"
claude "Work only on: Local Knowledge Bases: Common mistakes and risk control. Explain
the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-13
claude "Work only on: Local Knowledge Bases: Common mistakes and risk control. Explain
the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

13.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 13.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 13.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 13.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.

2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-13"
claude "Work only on: Local Knowledge Bases: Practice, deliverables, and review.
Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-13
claude "Work only on: Local Knowledge Bases: Practice, deliverables, and review.
Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.

- A small local case that can be repeated.

Chapter Summary

This chapter framed Local Knowledge Bases as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Memory System Anti-patterns.

Chapter 14: Memory System Anti-patterns

Part 3: Markdown, Memory, and Knowledge Bases

Main case language: C#

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Memory System Anti-patterns into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Memory System Anti-patterns is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a C# example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small C# project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-14"
claude "Work only on: Memory System Anti-patterns. Explain the plan, then show the
verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-14
claude "Work only on: Memory System Anti-patterns. Explain the plan, then show the
verification checklist."

using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Contents

- 14.1 Concepts and positioning
 - 14.1.1 Definition and use case
 - 14.1.2 Practical procedure
 - 14.1.3 Verification and boundary

- 14.2 Workflow and operating model
- 14.2.1 Definition and use case
- 14.2.2 Practical procedure
- 14.2.3 Verification and boundary
- 14.3 Common mistakes and risk control
- 14.3.1 Definition and use case
- 14.3.2 Practical procedure
- 14.3.3 Verification and boundary
- 14.4 Practice, deliverables, and review
- 14.4.1 Definition and use case
- 14.4.2 Practical procedure
- 14.4.3 Verification and boundary

14.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 14.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 14.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 14.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-14"
claude "Work only on: Memory System Anti-patterns: Concepts and positioning. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-14
claude "Work only on: Memory System Anti-patterns: Concepts and positioning. Explain
```

the plan, then show the verification checklist."

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

14.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 14.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 14.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 14.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.

3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-14"
claude "Work only on: Memory System Anti-patterns: Workflow and operating model.
Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-14
claude "Work only on: Memory System Anti-patterns: Workflow and operating model.
Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

14.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 14.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 14.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 14.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-14"
claude "Work only on: Memory System Anti-patterns: Common mistakes and risk control.
Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-14
claude "Work only on: Memory System Anti-patterns: Common mistakes and risk control.
Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

14.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.

- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 14.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 14.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 14.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-14"
claude "Work only on: Memory System Anti-patterns: Practice, deliverables, and review.
Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-14
claude "Work only on: Memory System Anti-patterns: Practice, deliverables, and review.
Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.

- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed Memory System Anti-patterns as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Claude Code Prompting Fundamentals.

Chapter 15: Claude Code Prompting Fundamentals

Part 4: Prompting and Model Control

Main case language: C++

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Claude Code Prompting Fundamentals into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Claude Code Prompting Fundamentals is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a C++ example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small C++ project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-15"
claude "Work only on: Claude Code Prompting Fundamentals. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-15
claude "Work only on: Claude Code Prompting Fundamentals. Explain the plan, then show the verification checklist."

#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Contents

- 15.1 Concepts and positioning
 - 15.1.1 Definition and use case
 - 15.1.2 Practical procedure
 - 15.1.3 Verification and boundary

- 15.2 Workflow and operating model
- 15.2.1 Definition and use case
- 15.2.2 Practical procedure
- 15.2.3 Verification and boundary
- 15.3 Common mistakes and risk control
- 15.3.1 Definition and use case
- 15.3.2 Practical procedure
- 15.3.3 Verification and boundary
- 15.4 Practice, deliverables, and review
- 15.4.1 Definition and use case
- 15.4.2 Practical procedure
- 15.4.3 Verification and boundary

15.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 15.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 15.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 15.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-15"
claude "Work only on: Claude Code Prompting Fundamentals: Concepts and positioning.
Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-15
claude "Work only on: Claude Code Prompting Fundamentals: Concepts and positioning."
```

Explain the plan, then show the verification checklist."

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

15.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 15.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 15.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 15.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.

5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-15"
claude "Work only on: Claude Code Prompting Fundamentals: Workflow and operating
model. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-15
claude "Work only on: Claude Code Prompting Fundamentals: Workflow and operating
model. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

15.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 15.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 15.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 15.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-15"
claude "Work only on: Claude Code Prompting Fundamentals: Common mistakes and risk
control. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-15
claude "Work only on: Claude Code Prompting Fundamentals: Common mistakes and risk
control. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

15.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 15.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 15.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 15.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-15"
claude "Work only on: Claude Code Prompting Fundamentals: Practice, deliverables, and
review. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-15
claude "Work only on: Claude Code Prompting Fundamentals: Practice, deliverables, and
review. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format

Problem	Symptom	Fix
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed Claude Code Prompting Fundamentals as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Plan Mode and Exploration Mode.

Chapter 16: Plan Mode and Exploration Mode

Part 4: Prompting and Model Control

Main case language: Python

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Plan Mode and Exploration Mode into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Plan Mode and Exploration Mode is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a Python example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small Python project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-16"
claude "Work only on: Plan Mode and Exploration Mode. Explain the plan, then show the
verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-16
claude "Work only on: Plan Mode and Exploration Mode. Explain the plan, then show the
verification checklist."

from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Contents

- 16.1 Concepts and positioning
 - 16.1.1 Definition and use case
 - 16.1.2 Practical procedure
 - 16.1.3 Verification and boundary
- 16.2 Workflow and operating model
 - 16.2.1 Definition and use case
 - 16.2.2 Practical procedure

- 16.2.3 Verification and boundary
- 16.3 Common mistakes and risk control
- 16.3.1 Definition and use case
- 16.3.2 Practical procedure
- 16.3.3 Verification and boundary
- 16.4 Practice, deliverables, and review
- 16.4.1 Definition and use case
- 16.4.2 Practical procedure
- 16.4.3 Verification and boundary

16.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 16.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 16.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 16.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-16"
claude "Work only on: Plan Mode and Exploration Mode: Concepts and positioning.
Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-16
claude "Work only on: Plan Mode and Exploration Mode: Concepts and positioning.
Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()
```

```
def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

16.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 16.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 16.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 16.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-16"
claude "Work only on: Plan Mode and Exploration Mode: Workflow and operating model.
Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-16
claude "Work only on: Plan Mode and Exploration Mode: Workflow and operating model."
```

Explain the plan, then show the verification checklist."

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

16.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 16.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 16.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 16.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.


```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-16"
claude "Work only on: Plan Mode and Exploration Mode: Common mistakes and risk
control. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-16
claude "Work only on: Plan Mode and Exploration Mode: Common mistakes and risk
control. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

16.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 16.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 16.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 16.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.

2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-16"
claude "Work only on: Plan Mode and Exploration Mode: Practice, deliverables, and
review. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-16
claude "Work only on: Plan Mode and Exploration Mode: Practice, deliverables, and
review. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.

- A small local case that can be repeated.

Chapter Summary

This chapter framed Plan Mode and Exploration Mode as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Taming Opus 4.7: Models, Effort, and Thinking Budget.

Chapter 17: Taming Opus 4.7: Models, Effort, and Thinking Budget

Part 4: Prompting and Model Control

Main case language: C#

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Taming Opus 4.7: Models, Effort, and Thinking Budget into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Taming Opus 4.7: Models, Effort, and Thinking Budget is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a C# example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small C# project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-17"
claude "Work only on: Taming Opus 4.7: Models, Effort, and Thinking Budget. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-17
claude "Work only on: Taming Opus 4.7: Models, Effort, and Thinking Budget. Explain the plan, then show the verification checklist."

using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) || Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Contents

- 17.1 Concepts and positioning
- 17.1.1 Definition and use case

- 17.1.2 Practical procedure
- 17.1.3 Verification and boundary
- 17.2 Workflow and operating model
- 17.2.1 Definition and use case
- 17.2.2 Practical procedure
- 17.2.3 Verification and boundary
- 17.3 Common mistakes and risk control
- 17.3.1 Definition and use case
- 17.3.2 Practical procedure
- 17.3.3 Verification and boundary
- 17.4 Practice, deliverables, and review
- 17.4.1 Definition and use case
- 17.4.2 Practical procedure
- 17.4.3 Verification and boundary

17.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 17.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 17.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 17.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-17"
claude "Work only on: Taming Opus 4.7: Models, Effort, and Thinking Budget: Concepts"
```

```
and positioning. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-17
claude "Work only on: Taming Opus 4.7: Models, Effort, and Thinking Budget: Concepts
and positioning. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

17.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 17.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 17.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 17.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-17"
claude "Work only on: Taming Opus 4.7: Models, Effort, and Thinking Budget: Workflow
and operating model. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-17
claude "Work only on: Taming Opus 4.7: Models, Effort, and Thinking Budget: Workflow
and operating model. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

17.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an

engineering behavior rather than a label.

Important subtopics:

- 17.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 17.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 17.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-17"
claude "Work only on: Taming Opus 4.7: Models, Effort, and Thinking Budget: Common
mistakes and risk control. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-17
claude "Work only on: Taming Opus 4.7: Models, Effort, and Thinking Budget: Common
mistakes and risk control. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

17.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 17.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 17.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 17.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-17"
claude "Work only on: Taming Opus 4.7: Models, Effort, and Thinking Budget: Practice,
deliverables, and review. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-17
claude "Work only on: Taming Opus 4.7: Models, Effort, and Thinking Budget: Practice,
deliverables, and review. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed Taming Opus 4.7: Models, Effort, and Thinking Budget as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Prompt Orchestration for Complex Engineering Tasks.

Chapter 18: Prompt Orchestration for Complex Engineering Tasks

Part 4: Prompting and Model Control

Main case language: C++

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Prompt Orchestration for Complex Engineering Tasks into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Prompt Orchestration for Complex Engineering Tasks is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a C++ example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small C++ project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-18"
claude "Work only on: Prompt Orchestration for Complex Engineering Tasks. Explain the
plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-18
claude "Work only on: Prompt Orchestration for Complex Engineering Tasks. Explain the
plan, then show the verification checklist."

#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Contents

- 18.1 Concepts and positioning
 - 18.1.1 Definition and use case
 - 18.1.2 Practical procedure
 - 18.1.3 Verification and boundary

- 18.2 Workflow and operating model
- 18.2.1 Definition and use case
- 18.2.2 Practical procedure
- 18.2.3 Verification and boundary
- 18.3 Common mistakes and risk control
- 18.3.1 Definition and use case
- 18.3.2 Practical procedure
- 18.3.3 Verification and boundary
- 18.4 Practice, deliverables, and review
- 18.4.1 Definition and use case
- 18.4.2 Practical procedure
- 18.4.3 Verification and boundary

18.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 18.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 18.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 18.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-18"
claude "Work only on: Prompt Orchestration for Complex Engineering Tasks: Concepts and
positioning. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-18
claude "Work only on: Prompt Orchestration for Complex Engineering Tasks: Concepts and
```

positioning. Explain the plan, then show the verification checklist."

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

18.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 18.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 18.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 18.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.

5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-18"
claude "Work only on: Prompt Orchestration for Complex Engineering Tasks: Workflow and
operating model. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-18
claude "Work only on: Prompt Orchestration for Complex Engineering Tasks: Workflow and
operating model. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

18.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 18.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 18.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 18.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-18"
claude "Work only on: Prompt Orchestration for Complex Engineering Tasks: Common
mistakes and risk control. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-18
claude "Work only on: Prompt Orchestration for Complex Engineering Tasks: Common
mistakes and risk control. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

18.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 18.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 18.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 18.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-18"
claude "Work only on: Prompt Orchestration for Complex Engineering Tasks: Practice,
deliverables, and review. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-18
claude "Work only on: Prompt Orchestration for Complex Engineering Tasks: Practice,
deliverables, and review. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format

Problem	Symptom	Fix
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed Prompt Orchestration for Complex Engineering Tasks as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Context Windows and File Selection.

Chapter 19: Context Windows and File Selection

Part 5: Context, Tokens, and Cost
Main case language: Python
Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Context Windows and File Selection into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Context Windows and File Selection is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a Python example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small Python project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-19"
claude "Work only on: Context Windows and File Selection. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-19
claude "Work only on: Context Windows and File Selection. Explain the plan, then show the verification checklist."

from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Contents

- 19.1 Concepts and positioning
 - 19.1.1 Definition and use case
 - 19.1.2 Practical procedure
 - 19.1.3 Verification and boundary
- 19.2 Workflow and operating model

- 19.2.1 Definition and use case
- 19.2.2 Practical procedure
- 19.2.3 Verification and boundary
- 19.3 Common mistakes and risk control
- 19.3.1 Definition and use case
- 19.3.2 Practical procedure
- 19.3.3 Verification and boundary
- 19.4 Practice, deliverables, and review
- 19.4.1 Definition and use case
- 19.4.2 Practical procedure
- 19.4.3 Verification and boundary

19.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 19.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 19.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 19.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-19"
claude "Work only on: Context Windows and File Selection: Concepts and positioning.
Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-19
claude "Work only on: Context Windows and File Selection: Concepts and positioning.
Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

19.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 19.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 19.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 19.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-19"
```

```
claude "Work only on: Context Windows and File Selection: Workflow and operating model. Explain the plan, then show the verification checklist."
```

```
# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-19
claude "Work only on: Context Windows and File Selection: Workflow and operating model. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

19.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 19.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 19.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 19.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.

3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-19"
claude "Work only on: Context Windows and File Selection: Common mistakes and risk
control. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-19
claude "Work only on: Context Windows and File Selection: Common mistakes and risk
control. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

19.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 19.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 19.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 19.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-19"
claude "Work only on: Context Windows and File Selection: Practice, deliverables, and
review. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-19
claude "Work only on: Context Windows and File Selection: Practice, deliverables, and
review. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed Context Windows and File Selection as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Compaction, Recovery, and Long-session Management.

Chapter 20: Compaction, Recovery, and Long-session Management

Part 5: Context, Tokens, and Cost

Main case language: C#

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Compaction, Recovery, and Long-session Management into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Compaction, Recovery, and Long-session Management is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a C# example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small C# project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-20"
claude "Work only on: Compaction, Recovery, and Long-session Management. Explain the
plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-20
claude "Work only on: Compaction, Recovery, and Long-session Management. Explain the
plan, then show the verification checklist."

using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Contents

- 20.1 Concepts and positioning
- 20.1.1 Definition and use case

- 20.1.2 Practical procedure
- 20.1.3 Verification and boundary
- 20.2 Workflow and operating model
- 20.2.1 Definition and use case
- 20.2.2 Practical procedure
- 20.2.3 Verification and boundary
- 20.3 Common mistakes and risk control
- 20.3.1 Definition and use case
- 20.3.2 Practical procedure
- 20.3.3 Verification and boundary
- 20.4 Practice, deliverables, and review
- 20.4.1 Definition and use case
- 20.4.2 Practical procedure
- 20.4.3 Verification and boundary

20.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 20.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 20.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 20.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-20"
claude "Work only on: Compaction, Recovery, and Long-session Management: Concepts and
```

```
positioning. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-20
claude "Work only on: Compaction, Recovery, and Long-session Management: Concepts and
positioning. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

20.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 20.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 20.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 20.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-20"
claude "Work only on: Compaction, Recovery, and Long-session Management: Workflow and
operating model. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-20
claude "Work only on: Compaction, Recovery, and Long-session Management: Workflow and
operating model. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

20.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an

engineering behavior rather than a label.

Important subtopics:

- 20.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 20.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 20.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-20"
claude "Work only on: Compaction, Recovery, and Long-session Management: Common
mistakes and risk control. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-20
claude "Work only on: Compaction, Recovery, and Long-session Management: Common
mistakes and risk control. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

20.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 20.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 20.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 20.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-20"
claude "Work only on: Compaction, Recovery, and Long-session Management: Practice,
deliverables, and review. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-20
claude "Work only on: Compaction, Recovery, and Long-session Management: Practice,
deliverables, and review. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed Compaction, Recovery, and Long-session Management as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Token Saving and Cost Optimization.

Chapter 21: Token Saving and Cost Optimization

Part 5: Context, Tokens, and Cost

Main case language: C++

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Token Saving and Cost Optimization into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Token Saving and Cost Optimization is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a C++ example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small C++ project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-21
claude "Work only on: Token Saving and Cost Optimization. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-21
claude "Work only on: Token Saving and Cost Optimization. Explain the plan, then show the verification checklist."

#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Contents

- 21.1 Concepts and positioning
 - 21.1.1 Definition and use case
 - 21.1.2 Practical procedure
 - 21.1.3 Verification and boundary

- 21.2 Workflow and operating model
- 21.2.1 Definition and use case
- 21.2.2 Practical procedure
- 21.2.3 Verification and boundary
- 21.3 Common mistakes and risk control
- 21.3.1 Definition and use case
- 21.3.2 Practical procedure
- 21.3.3 Verification and boundary
- 21.4 Practice, deliverables, and review
- 21.4.1 Definition and use case
- 21.4.2 Practical procedure
- 21.4.3 Verification and boundary

21.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 21.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 21.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 21.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-21"
claude "Work only on: Token Saving and Cost Optimization: Concepts and positioning.
Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-21
claude "Work only on: Token Saving and Cost Optimization: Concepts and positioning."
```

Explain the plan, then show the verification checklist."

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

21.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 21.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 21.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 21.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.

5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-21"
claude "Work only on: Token Saving and Cost Optimization: Workflow and operating
model. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-21
claude "Work only on: Token Saving and Cost Optimization: Workflow and operating
model. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

21.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 21.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 21.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 21.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-21"
claude "Work only on: Token Saving and Cost Optimization: Common mistakes and risk
control. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-21
claude "Work only on: Token Saving and Cost Optimization: Common mistakes and risk
control. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

21.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 21.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 21.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 21.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-21"
claude "Work only on: Token Saving and Cost Optimization: Practice, deliverables, and
review. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-21
claude "Work only on: Token Saving and Cost Optimization: Practice, deliverables, and
review. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format

Problem	Symptom	Fix
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed Token Saving and Cost Optimization as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Skills Fundamentals.

Chapter 22: Skills Fundamentals

Part 6: Extension Systems

Main case language: Python

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Skills Fundamentals into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Skills Fundamentals is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a Python example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small Python project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-22"
claude "Work only on: Skills Fundamentals. Explain the plan, then show the
verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-22
claude "Work only on: Skills Fundamentals. Explain the plan, then show the
verification checklist."

from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Contents

- 22.1 Concepts and positioning
 - 22.1.1 Definition and use case
 - 22.1.2 Practical procedure
 - 22.1.3 Verification and boundary
- 22.2 Workflow and operating model
 - 22.2.1 Definition and use case
 - 22.2.2 Practical procedure

- 22.2.3 Verification and boundary
- 22.3 Common mistakes and risk control
- 22.3.1 Definition and use case
- 22.3.2 Practical procedure
- 22.3.3 Verification and boundary
- 22.4 Practice, deliverables, and review
- 22.4.1 Definition and use case
- 22.4.2 Practical procedure
- 22.4.3 Verification and boundary

22.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 22.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 22.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 22.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-22"
claude "Work only on: Skills Fundamentals: Concepts and positioning. Explain the plan,
then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-22
claude "Work only on: Skills Fundamentals: Concepts and positioning. Explain the plan,
then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()
```

```
def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

22.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 22.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 22.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 22.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-22"
claude "Work only on: Skills Fundamentals: Workflow and operating model. Explain the
plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-22
claude "Work only on: Skills Fundamentals: Workflow and operating model. Explain the
```

plan, then show the verification checklist."

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

22.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 22.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 22.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 22.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.


```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-22"
claude "Work only on: Skills Fundamentals: Common mistakes and risk control. Explain
the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-22
claude "Work only on: Skills Fundamentals: Common mistakes and risk control. Explain
the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

22.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 22.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 22.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 22.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.

2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-22"
claude "Work only on: Skills Fundamentals: Practice, deliverables, and review. Explain
the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-22
claude "Work only on: Skills Fundamentals: Practice, deliverables, and review. Explain
the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.

- A small local case that can be repeated.

Chapter Summary

This chapter framed Skills Fundamentals as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Custom Commands and Task-oriented Skills.

Chapter 23: Custom Commands and Task-oriented Skills

Part 6: Extension Systems

Main case language: C#

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Custom Commands and Task-oriented Skills into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Custom Commands and Task-oriented Skills is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a C# example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small C# project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-23"
claude "Work only on: Custom Commands and Task-oriented Skills. Explain the plan, then
show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-23
claude "Work only on: Custom Commands and Task-oriented Skills. Explain the plan, then
show the verification checklist."

using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Contents

- 23.1 Concepts and positioning
- 23.1.1 Definition and use case

- 23.1.2 Practical procedure
- 23.1.3 Verification and boundary
- 23.2 Workflow and operating model
- 23.2.1 Definition and use case
- 23.2.2 Practical procedure
- 23.2.3 Verification and boundary
- 23.3 Common mistakes and risk control
- 23.3.1 Definition and use case
- 23.3.2 Practical procedure
- 23.3.3 Verification and boundary
- 23.4 Practice, deliverables, and review
- 23.4.1 Definition and use case
- 23.4.2 Practical procedure
- 23.4.3 Verification and boundary

23.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 23.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 23.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 23.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-23"
claude "Work only on: Custom Commands and Task-oriented Skills: Concepts and
```

```
positioning. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-23
claude "Work only on: Custom Commands and Task-oriented Skills: Concepts and
positioning. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

23.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 23.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 23.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 23.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-23"
claude "Work only on: Custom Commands and Task-oriented Skills: Workflow and operating
model. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-23
claude "Work only on: Custom Commands and Task-oriented Skills: Workflow and operating
model. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

23.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an

engineering behavior rather than a label.

Important subtopics:

- 23.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 23.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 23.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-23"
claude "Work only on: Custom Commands and Task-oriented Skills: Common mistakes and
risk control. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-23
claude "Work only on: Custom Commands and Task-oriented Skills: Common mistakes and
risk control. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

23.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 23.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 23.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 23.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-23"
claude "Work only on: Custom Commands and Task-oriented Skills: Practice,
deliverables, and review. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-23
claude "Work only on: Custom Commands and Task-oriented Skills: Practice,
deliverables, and review. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed Custom Commands and Task-oriented Skills as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with SubAgents Fundamentals.

Chapter 24: SubAgents Fundamentals

Part 6: Extension Systems

Main case language: C++

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of SubAgents Fundamentals into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what SubAgents Fundamentals is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a C++ example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small C++ project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-24"
claude "Work only on: SubAgents Fundamentals. Explain the plan, then show the
verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-24
claude "Work only on: SubAgents Fundamentals. Explain the plan, then show the
verification checklist."

#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Contents

- 24.1 Concepts and positioning
 - 24.1.1 Definition and use case
 - 24.1.2 Practical procedure
 - 24.1.3 Verification and boundary
- 24.2 Workflow and operating model
 - 24.2.1 Definition and use case

- 24.2.2 Practical procedure
- 24.2.3 Verification and boundary
- 24.3 Common mistakes and risk control
- 24.3.1 Definition and use case
- 24.3.2 Practical procedure
- 24.3.3 Verification and boundary
- 24.4 Practice, deliverables, and review
- 24.4.1 Definition and use case
- 24.4.2 Practical procedure
- 24.4.3 Verification and boundary

24.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 24.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 24.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 24.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-24"
claude "Work only on: SubAgents Fundamentals: Concepts and positioning. Explain the
plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-24
claude "Work only on: SubAgents Fundamentals: Concepts and positioning. Explain the
plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

24.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 24.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 24.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 24.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-24"
```

```
claude "Work only on: SubAgents Fundamentals: Workflow and operating model. Explain the plan, then show the verification checklist."
```

```
# macOS / Linux
```

```
cd ~/claude-code-handbook/book/examples/chapter-24
```

```
claude "Work only on: SubAgents Fundamentals: Workflow and operating model. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

24.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 24.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 24.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 24.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.

2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-24"
claude "Work only on: SubAgents Fundamentals: Common mistakes and risk control.
Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-24
claude "Work only on: SubAgents Fundamentals: Common mistakes and risk control.
Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

24.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 24.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.

- 24.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 24.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-24"
claude "Work only on: SubAgents Fundamentals: Practice, deliverables, and review.
Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-24
claude "Work only on: SubAgents Fundamentals: Practice, deliverables, and review.
Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context

Problem	Symptom	Fix
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed SubAgents Fundamentals as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Multi-Agent Collaboration.

Chapter 25: Multi-Agent Collaboration

Part 6: Extension Systems

Main case language: Python

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Multi-Agent Collaboration into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Multi-Agent Collaboration is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a Python example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small Python project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-25"
claude "Work only on: Multi-Agent Collaboration. Explain the plan, then show the
verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-25
claude "Work only on: Multi-Agent Collaboration. Explain the plan, then show the
verification checklist."

from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Contents

- 25.1 Concepts and positioning
 - 25.1.1 Definition and use case
 - 25.1.2 Practical procedure
 - 25.1.3 Verification and boundary
- 25.2 Workflow and operating model
 - 25.2.1 Definition and use case
 - 25.2.2 Practical procedure

- 25.2.3 Verification and boundary
- 25.3 Common mistakes and risk control
- 25.3.1 Definition and use case
- 25.3.2 Practical procedure
- 25.3.3 Verification and boundary
- 25.4 Practice, deliverables, and review
- 25.4.1 Definition and use case
- 25.4.2 Practical procedure
- 25.4.3 Verification and boundary

25.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 25.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 25.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 25.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-25"
claude "Work only on: Multi-Agent Collaboration: Concepts and positioning. Explain the
plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-25
claude "Work only on: Multi-Agent Collaboration: Concepts and positioning. Explain the
plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()
```

```
def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

25.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 25.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 25.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 25.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-25"
claude "Work only on: Multi-Agent Collaboration: Workflow and operating model. Explain
the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-25
claude "Work only on: Multi-Agent Collaboration: Workflow and operating model. Explain
```

the plan, then show the verification checklist."

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

25.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 25.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 25.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 25.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.


```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-25"
claude "Work only on: Multi-Agent Collaboration: Common mistakes and risk control.
Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-25
claude "Work only on: Multi-Agent Collaboration: Common mistakes and risk control.
Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

25.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 25.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 25.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 25.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.

2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-25"
claude "Work only on: Multi-Agent Collaboration: Practice, deliverables, and review.
Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-25
claude "Work only on: Multi-Agent Collaboration: Practice, deliverables, and review.
Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.

- A small local case that can be repeated.

Chapter Summary

This chapter framed Multi-Agent Collaboration as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Hooks for Event-driven Automation.

Chapter 26: Hooks for Event-driven Automation

Part 6: Extension Systems

Main case language: C#

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Hooks for Event-driven Automation into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Hooks for Event-driven Automation is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a C# example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small C# project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-26"
claude "Work only on: Hooks for Event-driven Automation. Explain the plan, then show
the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-26
claude "Work only on: Hooks for Event-driven Automation. Explain the plan, then show
the verification checklist."

using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Contents

- 26.1 Concepts and positioning
 - 26.1.1 Definition and use case
 - 26.1.2 Practical procedure
 - 26.1.3 Verification and boundary

- 26.2 Workflow and operating model
- 26.2.1 Definition and use case
- 26.2.2 Practical procedure
- 26.2.3 Verification and boundary
- 26.3 Common mistakes and risk control
- 26.3.1 Definition and use case
- 26.3.2 Practical procedure
- 26.3.3 Verification and boundary
- 26.4 Practice, deliverables, and review
- 26.4.1 Definition and use case
- 26.4.2 Practical procedure
- 26.4.3 Verification and boundary

26.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 26.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 26.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 26.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-26"
claude "Work only on: Hooks for Event-driven Automation: Concepts and positioning.
Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-26
claude "Work only on: Hooks for Event-driven Automation: Concepts and positioning."
```

Explain the plan, then show the verification checklist."

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

26.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 26.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 26.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 26.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.

3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-26"
claude "Work only on: Hooks for Event-driven Automation: Workflow and operating model.
  Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-26
claude "Work only on: Hooks for Event-driven Automation: Workflow and operating model.
  Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

26.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 26.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 26.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 26.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-26"
claude "Work only on: Hooks for Event-driven Automation: Common mistakes and risk
control. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-26
claude "Work only on: Hooks for Event-driven Automation: Common mistakes and risk
control. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

26.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.

- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 26.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 26.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 26.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code   \book\examples\chapter-26"
claude "Work only on: Hooks for Event-driven Automation: Practice, deliverables, and
review. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-26
claude "Work only on: Hooks for Event-driven Automation: Practice, deliverables, and
review. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.

- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed Hooks for Event-driven Automation as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with MCP Fundamentals and External Tool Integration.

Chapter 27: MCP Fundamentals and External Tool Integration

Part 6: Extension Systems

Main case language: C++

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of MCP Fundamentals and External Tool Integration into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what MCP Fundamentals and External Tool Integration is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a C++ example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small C++ project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-27"
claude "Work only on: MCP Fundamentals and External Tool Integration. Explain the
plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-27
claude "Work only on: MCP Fundamentals and External Tool Integration. Explain the
plan, then show the verification checklist."

#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Contents

- 27.1 Concepts and positioning
 - 27.1.1 Definition and use case
 - 27.1.2 Practical procedure
 - 27.1.3 Verification and boundary

- 27.2 Workflow and operating model
- 27.2.1 Definition and use case
- 27.2.2 Practical procedure
- 27.2.3 Verification and boundary
- 27.3 Common mistakes and risk control
- 27.3.1 Definition and use case
- 27.3.2 Practical procedure
- 27.3.3 Verification and boundary
- 27.4 Practice, deliverables, and review
- 27.4.1 Definition and use case
- 27.4.2 Practical procedure
- 27.4.3 Verification and boundary

27.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 27.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 27.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 27.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code   \book\examples\chapter-27"
claude "Work only on: MCP Fundamentals and External Tool Integration: Concepts and
positioning. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-27
claude "Work only on: MCP Fundamentals and External Tool Integration: Concepts and
```

positioning. Explain the plan, then show the verification checklist."

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

27.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 27.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 27.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 27.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.

5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-27"
claude "Work only on: MCP Fundamentals and External Tool Integration: Workflow and
operating model. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-27
claude "Work only on: MCP Fundamentals and External Tool Integration: Workflow and
operating model. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

27.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 27.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 27.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 27.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-27"
claude "Work only on: MCP Fundamentals and External Tool Integration: Common mistakes
and risk control. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-27
claude "Work only on: MCP Fundamentals and External Tool Integration: Common mistakes
and risk control. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

27.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 27.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 27.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 27.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-27"
claude "Work only on: MCP Fundamentals and External Tool Integration: Practice,
deliverables, and review. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-27
claude "Work only on: MCP Fundamentals and External Tool Integration: Practice,
deliverables, and review. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format

Problem	Symptom	Fix
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed MCP Fundamentals and External Tool Integration as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with The Plugins System.

Chapter 28: The Plugins System

Part 6: Extension Systems

Main case language: Python

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of The Plugins System into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what The Plugins System is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a Python example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small Python project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-28"
claude "Work only on: The Plugins System. Explain the plan, then show the verification
checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-28
claude "Work only on: The Plugins System. Explain the plan, then show the verification
checklist."

from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Contents

- 28.1 Concepts and positioning
 - 28.1.1 Definition and use case
 - 28.1.2 Practical procedure
 - 28.1.3 Verification and boundary
- 28.2 Workflow and operating model
 - 28.2.1 Definition and use case
 - 28.2.2 Practical procedure

- 28.2.3 Verification and boundary
- 28.3 Common mistakes and risk control
- 28.3.1 Definition and use case
- 28.3.2 Practical procedure
- 28.3.3 Verification and boundary
- 28.4 Practice, deliverables, and review
- 28.4.1 Definition and use case
- 28.4.2 Practical procedure
- 28.4.3 Verification and boundary

28.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 28.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 28.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 28.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-28"
claude "Work only on: The Plugins System: Concepts and positioning. Explain the plan,
then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-28
claude "Work only on: The Plugins System: Concepts and positioning. Explain the plan,
then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()
```

```
def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

28.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 28.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 28.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 28.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-28"
claude "Work only on: The Plugins System: Workflow and operating model. Explain the
plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-28
claude "Work only on: The Plugins System: Workflow and operating model. Explain the
```

plan, then show the verification checklist."

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

28.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 28.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 28.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 28.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.


```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-28"
claude "Work only on: The Plugins System: Common mistakes and risk control. Explain
the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-28
claude "Work only on: The Plugins System: Common mistakes and risk control. Explain
the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

28.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 28.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 28.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 28.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.

2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-28"
claude "Work only on: The Plugins System: Practice, deliverables, and review. Explain
the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-28
claude "Work only on: The Plugins System: Practice, deliverables, and review. Explain
the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.

- A small local case that can be repeated.

Chapter Summary

This chapter framed The Plugins System as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Tools and Rules.

Chapter 29: Tools and Rules

Part 6: Extension Systems

Main case language: C#

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Tools and Rules into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Tools and Rules is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a C# example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small C# project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-29"
claude "Work only on: Tools and Rules. Explain the plan, then show the verification
checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-29
claude "Work only on: Tools and Rules. Explain the plan, then show the verification
checklist."

using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Contents

- 29.1 Concepts and positioning
 - 29.1.1 Definition and use case
 - 29.1.2 Practical procedure
 - 29.1.3 Verification and boundary

- 29.2 Workflow and operating model
- 29.2.1 Definition and use case
- 29.2.2 Practical procedure
- 29.2.3 Verification and boundary
- 29.3 Common mistakes and risk control
- 29.3.1 Definition and use case
- 29.3.2 Practical procedure
- 29.3.3 Verification and boundary
- 29.4 Practice, deliverables, and review
- 29.4.1 Definition and use case
- 29.4.2 Practical procedure
- 29.4.3 Verification and boundary

29.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 29.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 29.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 29.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-29"
claude "Work only on: Tools and Rules: Concepts and positioning. Explain the plan,
then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-29
claude "Work only on: Tools and Rules: Concepts and positioning. Explain the plan,
```

then show the verification checklist."

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

29.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 29.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 29.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 29.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.

3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-29"
claude "Work only on: Tools and Rules: Workflow and operating model. Explain the plan,
then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-29
claude "Work only on: Tools and Rules: Workflow and operating model. Explain the plan,
then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

29.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 29.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 29.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 29.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-29"
claude "Work only on: Tools and Rules: Common mistakes and risk control. Explain the
plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-29
claude "Work only on: Tools and Rules: Common mistakes and risk control. Explain the
plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

29.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.

- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 29.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 29.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 29.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-29"
claude "Work only on: Tools and Rules: Practice, deliverables, and review. Explain the
plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-29
claude "Work only on: Tools and Rules: Practice, deliverables, and review. Explain the
plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.

- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed Tools and Rules as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Combination Patterns for Skill, Agent, MCP, and Plugin.

Chapter 30: Combination Patterns for Skill, Agent, MCP, and Plugin

Part 6: Extension Systems

Main case language: C++

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Combination Patterns for Skill, Agent, MCP, and Plugin into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Combination Patterns for Skill, Agent, MCP, and Plugin is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a C++ example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small C++ project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-30"
claude "Work only on: Combination Patterns for Skill, Agent, MCP, and Plugin. Explain
the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-30
claude "Work only on: Combination Patterns for Skill, Agent, MCP, and Plugin. Explain
the plan, then show the verification checklist."

#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Contents

- 30.1 Concepts and positioning
 - 30.1.1 Definition and use case
 - 30.1.2 Practical procedure
 - 30.1.3 Verification and boundary

- 30.2 Workflow and operating model
- 30.2.1 Definition and use case
- 30.2.2 Practical procedure
- 30.2.3 Verification and boundary
- 30.3 Common mistakes and risk control
- 30.3.1 Definition and use case
- 30.3.2 Practical procedure
- 30.3.3 Verification and boundary
- 30.4 Practice, deliverables, and review
- 30.4.1 Definition and use case
- 30.4.2 Practical procedure
- 30.4.3 Verification and boundary

30.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 30.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 30.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 30.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code   \book\examples\chapter-30"
claude "Work only on: Combination Patterns for Skill, Agent, MCP, and Plugin: Concepts
and positioning. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-30
claude "Work only on: Combination Patterns for Skill, Agent, MCP, and Plugin: Concepts
```

and positioning. Explain the plan, then show the verification checklist."

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

30.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 30.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 30.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 30.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.

5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-30"
claude "Work only on: Combination Patterns for Skill, Agent, MCP, and Plugin: Workflow
and operating model. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-30
claude "Work only on: Combination Patterns for Skill, Agent, MCP, and Plugin: Workflow
and operating model. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

30.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 30.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 30.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 30.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-30"
claude "Work only on: Combination Patterns for Skill, Agent, MCP, and Plugin: Common
mistakes and risk control. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-30
claude "Work only on: Combination Patterns for Skill, Agent, MCP, and Plugin: Common
mistakes and risk control. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

30.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 30.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 30.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 30.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-30"
claude "Work only on: Combination Patterns for Skill, Agent, MCP, and Plugin:
Practice, deliverables, and review. Explain the plan, then show the verification
checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-30
claude "Work only on: Combination Patterns for Skill, Agent, MCP, and Plugin:
Practice, deliverables, and review. Explain the plan, then show the verification
checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format

Problem	Symptom	Fix
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed Combination Patterns for Skill, Agent, MCP, and Plugin as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Mini Project: Building an Extension System.

Chapter 31: Mini Project: Building an Extension System

Part 6: Extension Systems

Main case language: Python

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Mini Project: Building an Extension System into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Mini Project: Building an Extension System is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a Python example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small Python project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-31"
claude "Work only on: Mini Project: Building an Extension System. Explain the plan,
then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-31
claude "Work only on: Mini Project: Building an Extension System. Explain the plan,
then show the verification checklist."

from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Contents

- 31.1 Concepts and positioning
 - 31.1.1 Definition and use case
 - 31.1.2 Practical procedure
 - 31.1.3 Verification and boundary
- 31.2 Workflow and operating model

- 31.2.1 Definition and use case
- 31.2.2 Practical procedure
- 31.2.3 Verification and boundary
- 31.3 Common mistakes and risk control
- 31.3.1 Definition and use case
- 31.3.2 Practical procedure
- 31.3.3 Verification and boundary
- 31.4 Practice, deliverables, and review
- 31.4.1 Definition and use case
- 31.4.2 Practical procedure
- 31.4.3 Verification and boundary

31.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 31.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 31.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 31.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-31"
claude "Work only on: Mini Project: Building an Extension System: Concepts and
positioning. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-31
claude "Work only on: Mini Project: Building an Extension System: Concepts and
positioning. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

31.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 31.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 31.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 31.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-31"
```

```
claude "Work only on: Mini Project: Building an Extension System: Workflow and operating model. Explain the plan, then show the verification checklist."
```

```
# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-31
claude "Work only on: Mini Project: Building an Extension System: Workflow and operating model. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

31.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 31.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 31.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 31.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.

3. Require file paths, commands, logs, or citations as evidence.

4. Ask for a plan before any risky action.

5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-31"
claude "Work only on: Mini Project: Building an Extension System: Common mistakes and
risk control. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-31
claude "Work only on: Mini Project: Building an Extension System: Common mistakes and
risk control. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

31.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 31.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 31.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 31.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-31"
claude "Work only on: Mini Project: Building an Extension System: Practice,
deliverables, and review. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-31
claude "Work only on: Mini Project: Building an Extension System: Practice,
deliverables, and review. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed Mini Project: Building an Extension System as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Headless Mode and Command-line Automation.

Chapter 32: Headless Mode and Command-line Automation

Part 7: Automation and CI

Main case language: C#

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Headless Mode and Command-line Automation into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Headless Mode and Command-line Automation is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a C# example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small C# project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-32"
claude "Work only on: Headless Mode and Command-line Automation. Explain the plan,
then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-32
claude "Work only on: Headless Mode and Command-line Automation. Explain the plan,
then show the verification checklist."

using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Contents

- 32.1 Concepts and positioning
- 32.1.1 Definition and use case

- 32.1.2 Practical procedure
- 32.1.3 Verification and boundary
- 32.2 Workflow and operating model
- 32.2.1 Definition and use case
- 32.2.2 Practical procedure
- 32.2.3 Verification and boundary
- 32.3 Common mistakes and risk control
- 32.3.1 Definition and use case
- 32.3.2 Practical procedure
- 32.3.3 Verification and boundary
- 32.4 Practice, deliverables, and review
- 32.4.1 Definition and use case
- 32.4.2 Practical procedure
- 32.4.3 Verification and boundary

32.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 32.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 32.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 32.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-32"
claude "Work only on: Headless Mode and Command-line Automation: Concepts and
```

```
positioning. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-32
claude "Work only on: Headless Mode and Command-line Automation: Concepts and
positioning. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

32.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 32.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 32.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 32.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-32"
claude "Work only on: Headless Mode and Command-line Automation: Workflow and
operating model. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-32
claude "Work only on: Headless Mode and Command-line Automation: Workflow and
operating model. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

32.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an

engineering behavior rather than a label.

Important subtopics:

- 32.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 32.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 32.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-32"
claude "Work only on: Headless Mode and Command-line Automation: Common mistakes and
risk control. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-32
claude "Work only on: Headless Mode and Command-line Automation: Common mistakes and
risk control. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

32.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 32.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 32.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 32.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-32"
claude "Work only on: Headless Mode and Command-line Automation: Practice,
deliverables, and review. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-32
claude "Work only on: Headless Mode and Command-line Automation: Practice,
deliverables, and review. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed Headless Mode and Command-line Automation as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Agent SDK Introduction.

Chapter 33: Agent SDK Introduction

Part 7: Automation and CI

Main case language: C++

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Agent SDK Introduction into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Agent SDK Introduction is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a C++ example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small C++ project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-33"
claude "Work only on: Agent SDK Introduction. Explain the plan, then show the
verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-33
claude "Work only on: Agent SDK Introduction. Explain the plan, then show the
verification checklist."

#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Contents

- 33.1 Concepts and positioning
 - 33.1.1 Definition and use case
 - 33.1.2 Practical procedure
 - 33.1.3 Verification and boundary
- 33.2 Workflow and operating model
 - 33.2.1 Definition and use case

- 33.2.2 Practical procedure
- 33.2.3 Verification and boundary
- 33.3 Common mistakes and risk control
- 33.3.1 Definition and use case
- 33.3.2 Practical procedure
- 33.3.3 Verification and boundary
- 33.4 Practice, deliverables, and review
- 33.4.1 Definition and use case
- 33.4.2 Practical procedure
- 33.4.3 Verification and boundary

33.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 33.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 33.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 33.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-33"
claude "Work only on: Agent SDK Introduction: Concepts and positioning. Explain the
plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-33
claude "Work only on: Agent SDK Introduction: Concepts and positioning. Explain the
plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

33.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 33.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 33.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 33.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-33"
```

```
claude "Work only on: Agent SDK Introduction: Workflow and operating model. Explain the plan, then show the verification checklist."
```

```
# macOS / Linux
```

```
cd ~/claude-code-handbook/book/examples/chapter-33
```

```
claude "Work only on: Agent SDK Introduction: Workflow and operating model. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

33.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 33.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 33.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 33.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.

2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-33"
claude "Work only on: Agent SDK Introduction: Common mistakes and risk control.
Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-33
claude "Work only on: Agent SDK Introduction: Common mistakes and risk control.
Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

33.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 33.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.

- 33.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 33.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-33"
claude "Work only on: Agent SDK Introduction: Practice, deliverables, and review.
Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-33
claude "Work only on: Agent SDK Introduction: Practice, deliverables, and review.
Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context

Problem	Symptom	Fix
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed Agent SDK Introduction as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Routines, Scheduled Tasks, and Automated Workflows.

Chapter 34: Routines, Scheduled Tasks, and Automated Workflows

Part 7: Automation and CI

Main case language: Python

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Routines, Scheduled Tasks, and Automated Workflows into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Routines, Scheduled Tasks, and Automated Workflows is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a Python example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small Python project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-34"
claude "Work only on: Routines, Scheduled Tasks, and Automated Workflows. Explain the
plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-34
claude "Work only on: Routines, Scheduled Tasks, and Automated Workflows. Explain the
plan, then show the verification checklist."

from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Contents

- 34.1 Concepts and positioning
 - 34.1.1 Definition and use case
 - 34.1.2 Practical procedure
 - 34.1.3 Verification and boundary
- 34.2 Workflow and operating model

- 34.2.1 Definition and use case
- 34.2.2 Practical procedure
- 34.2.3 Verification and boundary
- 34.3 Common mistakes and risk control
- 34.3.1 Definition and use case
- 34.3.2 Practical procedure
- 34.3.3 Verification and boundary
- 34.4 Practice, deliverables, and review
- 34.4.1 Definition and use case
- 34.4.2 Practical procedure
- 34.4.3 Verification and boundary

34.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 34.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 34.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 34.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-34"
claude "Work only on: Routines, Scheduled Tasks, and Automated Workflows: Concepts and
positioning. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-34
claude "Work only on: Routines, Scheduled Tasks, and Automated Workflows: Concepts and
positioning. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

34.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 34.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 34.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 34.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-34"
```

```
claude "Work only on: Routines, Scheduled Tasks, and Automated Workflows: Workflow and operating model. Explain the plan, then show the verification checklist."
```

```
# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-34
claude "Work only on: Routines, Scheduled Tasks, and Automated Workflows: Workflow and operating model. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

34.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 34.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 34.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 34.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.

3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-34"
claude "Work only on: Routines, Scheduled Tasks, and Automated Workflows: Common
mistakes and risk control. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-34
claude "Work only on: Routines, Scheduled Tasks, and Automated Workflows: Common
mistakes and risk control. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

34.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 34.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 34.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 34.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-34"
claude "Work only on: Routines, Scheduled Tasks, and Automated Workflows: Practice,
deliverables, and review. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-34
claude "Work only on: Routines, Scheduled Tasks, and Automated Workflows: Practice,
deliverables, and review. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed Routines, Scheduled Tasks, and Automated Workflows as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with GitHub PR and CI Workflows.

Chapter 35: GitHub PR and CI Workflows

Part 7: Automation and CI

Main case language: C#

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of GitHub PR and CI Workflows into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what GitHub PR and CI Workflows is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a C# example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small C# project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-35"
claude "Work only on: GitHub PR and CI Workflows. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-35
claude "Work only on: GitHub PR and CI Workflows. Explain the plan, then show the verification checklist."

using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) || Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Contents

- 35.1 Concepts and positioning
 - 35.1.1 Definition and use case
 - 35.1.2 Practical procedure
 - 35.1.3 Verification and boundary

- 35.2 Workflow and operating model
- 35.2.1 Definition and use case
- 35.2.2 Practical procedure
- 35.2.3 Verification and boundary
- 35.3 Common mistakes and risk control
- 35.3.1 Definition and use case
- 35.3.2 Practical procedure
- 35.3.3 Verification and boundary
- 35.4 Practice, deliverables, and review
- 35.4.1 Definition and use case
- 35.4.2 Practical procedure
- 35.4.3 Verification and boundary

35.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 35.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 35.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 35.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-35"
claude "Work only on: GitHub PR and CI Workflows: Concepts and positioning. Explain
the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-35
claude "Work only on: GitHub PR and CI Workflows: Concepts and positioning. Explain
```

the plan, then show the verification checklist."

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

35.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 35.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 35.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 35.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.

3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-35"
claude "Work only on: GitHub PR and CI Workflows: Workflow and operating model.
Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-35
claude "Work only on: GitHub PR and CI Workflows: Workflow and operating model.
Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

35.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 35.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 35.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 35.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-35"
claude "Work only on: GitHub PR and CI Workflows: Common mistakes and risk control.
Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-35
claude "Work only on: GitHub PR and CI Workflows: Common mistakes and risk control.
Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

35.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.

- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 35.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 35.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 35.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-35"
claude "Work only on: GitHub PR and CI Workflows: Practice, deliverables, and review.
Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-35
claude "Work only on: GitHub PR and CI Workflows: Practice, deliverables, and review.
Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.

- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed GitHub PR and CI Workflows as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with IDE, Browser, and External Tool Integration.

Chapter 36: IDE, Browser, and External Tool Integration

Part 7: Automation and CI

Main case language: C++

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of IDE, Browser, and External Tool Integration into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what IDE, Browser, and External Tool Integration is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a C++ example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small C++ project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-36"
claude "Work only on: IDE, Browser, and External Tool Integration. Explain the plan,
then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-36
claude "Work only on: IDE, Browser, and External Tool Integration. Explain the plan,
then show the verification checklist."

#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Contents

- 36.1 Concepts and positioning
 - 36.1.1 Definition and use case
 - 36.1.2 Practical procedure
 - 36.1.3 Verification and boundary

- 36.2 Workflow and operating model
- 36.2.1 Definition and use case
- 36.2.2 Practical procedure
- 36.2.3 Verification and boundary
- 36.3 Common mistakes and risk control
- 36.3.1 Definition and use case
- 36.3.2 Practical procedure
- 36.3.3 Verification and boundary
- 36.4 Practice, deliverables, and review
- 36.4.1 Definition and use case
- 36.4.2 Practical procedure
- 36.4.3 Verification and boundary

36.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 36.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 36.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 36.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code   \book\examples\chapter-36"
claude "Work only on: IDE, Browser, and External Tool Integration: Concepts and
positioning. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-36
claude "Work only on: IDE, Browser, and External Tool Integration: Concepts and
```

positioning. Explain the plan, then show the verification checklist."

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

36.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 36.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 36.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 36.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.

5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-36"
claude "Work only on: IDE, Browser, and External Tool Integration: Workflow and
operating model. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-36
claude "Work only on: IDE, Browser, and External Tool Integration: Workflow and
operating model. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

36.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 36.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 36.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 36.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-36"
claude "Work only on: IDE, Browser, and External Tool Integration: Common mistakes and
risk control. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-36
claude "Work only on: IDE, Browser, and External Tool Integration: Common mistakes and
risk control. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

36.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 36.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 36.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 36.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-36"
claude "Work only on: IDE, Browser, and External Tool Integration: Practice,
deliverables, and review. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-36
claude "Work only on: IDE, Browser, and External Tool Integration: Practice,
deliverables, and review. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format

Problem	Symptom	Fix
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed IDE, Browser, and External Tool Integration as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Permissions and Safe Working Modes.

Chapter 37: Permissions and Safe Working Modes

Part 8: Security and Risk Control

Main case language: Python

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Permissions and Safe Working Modes into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Permissions and Safe Working Modes is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a Python example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small Python project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-37"
claude "Work only on: Permissions and Safe Working Modes. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-37
claude "Work only on: Permissions and Safe Working Modes. Explain the plan, then show the verification checklist."

from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Contents

- 37.1 Concepts and positioning
 - 37.1.1 Definition and use case
 - 37.1.2 Practical procedure
 - 37.1.3 Verification and boundary
- 37.2 Workflow and operating model

- 37.2.1 Definition and use case
- 37.2.2 Practical procedure
- 37.2.3 Verification and boundary
- 37.3 Common mistakes and risk control
- 37.3.1 Definition and use case
- 37.3.2 Practical procedure
- 37.3.3 Verification and boundary
- 37.4 Practice, deliverables, and review
- 37.4.1 Definition and use case
- 37.4.2 Practical procedure
- 37.4.3 Verification and boundary

37.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 37.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 37.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 37.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-37"
claude "Work only on: Permissions and Safe Working Modes: Concepts and positioning.
Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-37
claude "Work only on: Permissions and Safe Working Modes: Concepts and positioning.
Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

37.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 37.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 37.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 37.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-37"
```

```
claude "Work only on: Permissions and Safe Working Modes: Workflow and operating model. Explain the plan, then show the verification checklist."
```

```
# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-37
claude "Work only on: Permissions and Safe Working Modes: Workflow and operating model. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

37.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 37.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 37.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 37.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.

3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-37"
claude "Work only on: Permissions and Safe Working Modes: Common mistakes and risk
control. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-37
claude "Work only on: Permissions and Safe Working Modes: Common mistakes and risk
control. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

37.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 37.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 37.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 37.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-37"
claude "Work only on: Permissions and Safe Working Modes: Practice, deliverables, and
review. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-37
claude "Work only on: Permissions and Safe Working Modes: Practice, deliverables, and
review. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed Permissions and Safe Working Modes as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Sandboxes, Data Security, and Prompt Injection.

Chapter 38: Sandboxes, Data Security, and Prompt Injection

Part 8: Security and Risk Control

Main case language: C#

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Sandboxes, Data Security, and Prompt Injection into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Sandboxes, Data Security, and Prompt Injection is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a C# example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small C# project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-38"
claude "Work only on: Sandboxes, Data Security, and Prompt Injection. Explain the
plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-38
claude "Work only on: Sandboxes, Data Security, and Prompt Injection. Explain the
plan, then show the verification checklist."

using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Contents

- 38.1 Concepts and positioning
- 38.1.1 Definition and use case

- 38.1.2 Practical procedure
- 38.1.3 Verification and boundary
- 38.2 Workflow and operating model
- 38.2.1 Definition and use case
- 38.2.2 Practical procedure
- 38.2.3 Verification and boundary
- 38.3 Common mistakes and risk control
- 38.3.1 Definition and use case
- 38.3.2 Practical procedure
- 38.3.3 Verification and boundary
- 38.4 Practice, deliverables, and review
- 38.4.1 Definition and use case
- 38.4.2 Practical procedure
- 38.4.3 Verification and boundary

38.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 38.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 38.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 38.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-38"
claude "Work only on: Sandboxes, Data Security, and Prompt Injection: Concepts and
```

```
positioning. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-38
claude "Work only on: Sandboxes, Data Security, and Prompt Injection: Concepts and
positioning. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

38.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 38.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 38.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 38.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-38"
claude "Work only on: Sandboxes, Data Security, and Prompt Injection: Workflow and
operating model. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-38
claude "Work only on: Sandboxes, Data Security, and Prompt Injection: Workflow and
operating model. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

38.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an

engineering behavior rather than a label.

Important subtopics:

- 38.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 38.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 38.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-38"
claude "Work only on: Sandboxes, Data Security, and Prompt Injection: Common mistakes
and risk control. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-38
claude "Work only on: Sandboxes, Data Security, and Prompt Injection: Common mistakes
and risk control. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

38.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 38.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 38.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 38.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-38"
claude "Work only on: Sandboxes, Data Security, and Prompt Injection: Practice,
deliverables, and review. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-38
claude "Work only on: Sandboxes, Data Security, and Prompt Injection: Practice,
deliverables, and review. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed Sandboxes, Data Security, and Prompt Injection as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Git, Safe Releases, and Rollbacks.

Chapter 39: Git, Safe Releases, and Rollbacks

Part 8: Security and Risk Control

Main case language: C++

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Git, Safe Releases, and Rollbacks into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Git, Safe Releases, and Rollbacks is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a C++ example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small C++ project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-39"
claude "Work only on: Git, Safe Releases, and Rollbacks. Explain the plan, then show
the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-39
claude "Work only on: Git, Safe Releases, and Rollbacks. Explain the plan, then show
the verification checklist."

#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Contents

- 39.1 Concepts and positioning
 - 39.1.1 Definition and use case
 - 39.1.2 Practical procedure
 - 39.1.3 Verification and boundary
- 39.2 Workflow and operating model
 - 39.2.1 Definition and use case

- 39.2.2 Practical procedure
- 39.2.3 Verification and boundary
- 39.3 Common mistakes and risk control
- 39.3.1 Definition and use case
- 39.3.2 Practical procedure
- 39.3.3 Verification and boundary
- 39.4 Practice, deliverables, and review
- 39.4.1 Definition and use case
- 39.4.2 Practical procedure
- 39.4.3 Verification and boundary

39.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 39.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 39.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 39.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-39"
claude "Work only on: Git, Safe Releases, and Rollbacks: Concepts and positioning.
Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-39
claude "Work only on: Git, Safe Releases, and Rollbacks: Concepts and positioning.
Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

39.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 39.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 39.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 39.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-39"
```

```
claude "Work only on: Git, Safe Releases, and Rollbacks: Workflow and operating model.  
Explain the plan, then show the verification checklist."
```

```
# macOS / Linux  
cd ~/claude-code-handbook/book/examples/chapter-39  
claude "Work only on: Git, Safe Releases, and Rollbacks: Workflow and operating model.  
Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>  
#include <iostream>  
  
int main() {  
    std::filesystem::path target{"README.md"};  
    std::cout << "Path: " << std::filesystem::absolute(target) << "  
";  
    std::cout << "Exists: " << std::filesystem::exists(target) << "  
";  
    return 0;  
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

39.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 39.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 39.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 39.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.

2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-39"
claude "Work only on: Git, Safe Releases, and Rollbacks: Common mistakes and risk control. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-39
claude "Work only on: Git, Safe Releases, and Rollbacks: Common mistakes and risk control. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

39.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 39.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.

- 39.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 39.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-39"
claude "Work only on: Git, Safe Releases, and Rollbacks: Practice, deliverables, and review. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-39
claude "Work only on: Git, Safe Releases, and Rollbacks: Practice, deliverables, and review. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context

Problem	Symptom	Fix
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed Git, Safe Releases, and Rollbacks as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Production-grade Claude Code Usage Guidelines.

Chapter 40: Production-grade Claude Code Usage Guidelines

Part 8: Security and Risk Control
Main case language: Python
Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Production-grade Claude Code Usage Guidelines into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Production-grade Claude Code Usage Guidelines is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a Python example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small Python project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-40"
claude "Work only on: Production-grade Claude Code Usage Guidelines. Explain the plan,
then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-40
claude "Work only on: Production-grade Claude Code Usage Guidelines. Explain the plan,
then show the verification checklist."

from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Contents

- 40.1 Concepts and positioning
 - 40.1.1 Definition and use case
 - 40.1.2 Practical procedure
 - 40.1.3 Verification and boundary
- 40.2 Workflow and operating model

- 40.2.1 Definition and use case
- 40.2.2 Practical procedure
- 40.2.3 Verification and boundary
- 40.3 Common mistakes and risk control
- 40.3.1 Definition and use case
- 40.3.2 Practical procedure
- 40.3.3 Verification and boundary
- 40.4 Practice, deliverables, and review
- 40.4.1 Definition and use case
- 40.4.2 Practical procedure
- 40.4.3 Verification and boundary

40.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 40.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 40.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 40.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-40"
claude "Work only on: Production-grade Claude Code Usage Guidelines: Concepts and
positioning. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-40
claude "Work only on: Production-grade Claude Code Usage Guidelines: Concepts and
positioning. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

40.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 40.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 40.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 40.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-40"
```

```
claude "Work only on: Production-grade Claude Code Usage Guidelines: Workflow and operating model. Explain the plan, then show the verification checklist."
```

```
# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-40
claude "Work only on: Production-grade Claude Code Usage Guidelines: Workflow and operating model. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

40.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 40.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 40.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 40.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.

3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-40"
claude "Work only on: Production-grade Claude Code Usage Guidelines: Common mistakes
and risk control. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-40
claude "Work only on: Production-grade Claude Code Usage Guidelines: Common mistakes
and risk control. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

40.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 40.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 40.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 40.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-40"
claude "Work only on: Production-grade Claude Code Usage Guidelines: Practice,
deliverables, and review. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-40
claude "Work only on: Production-grade Claude Code Usage Guidelines: Practice,
deliverables, and review. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed Production-grade Claude Code Usage Guidelines as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Claude Code Internal Mechanics.

Chapter 41: Claude Code Internal Mechanics

Part 9: Advanced Harness Engineering

Main case language: C#

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Claude Code Internal Mechanics into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Claude Code Internal Mechanics is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a C# example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small C# project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-41"
claude "Work only on: Claude Code Internal Mechanics. Explain the plan, then show the
verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-41
claude "Work only on: Claude Code Internal Mechanics. Explain the plan, then show the
verification checklist."

using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Contents

- 41.1 Concepts and positioning
 - 41.1.1 Definition and use case
 - 41.1.2 Practical procedure
 - 41.1.3 Verification and boundary

- 41.2 Workflow and operating model
- 41.2.1 Definition and use case
- 41.2.2 Practical procedure
- 41.2.3 Verification and boundary
- 41.3 Common mistakes and risk control
- 41.3.1 Definition and use case
- 41.3.2 Practical procedure
- 41.3.3 Verification and boundary
- 41.4 Practice, deliverables, and review
- 41.4.1 Definition and use case
- 41.4.2 Practical procedure
- 41.4.3 Verification and boundary

41.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 41.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 41.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 41.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code   \book\examples\chapter-41"
claude "Work only on: Claude Code Internal Mechanics: Concepts and positioning.
Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-41
claude "Work only on: Claude Code Internal Mechanics: Concepts and positioning."
```

Explain the plan, then show the verification checklist."

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

41.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 41.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 41.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 41.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.

3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-41"
claude "Work only on: Claude Code Internal Mechanics: Workflow and operating model.
Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-41
claude "Work only on: Claude Code Internal Mechanics: Workflow and operating model.
Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

41.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 41.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 41.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 41.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-41"
claude "Work only on: Claude Code Internal Mechanics: Common mistakes and risk control. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-41
claude "Work only on: Claude Code Internal Mechanics: Common mistakes and risk control. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) || Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

41.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.

- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 41.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 41.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 41.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-41"
claude "Work only on: Claude Code Internal Mechanics: Practice, deliverables, and review. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-41
claude "Work only on: Claude Code Internal Mechanics: Practice, deliverables, and review. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) || Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.

- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed Claude Code Internal Mechanics as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Context Engineering, Caching, and Cost Engineering.

Chapter 42: Context Engineering, Caching, and Cost Engineering

Part 9: Advanced Harness Engineering

Main case language: C++

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Context Engineering, Caching, and Cost Engineering into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Context Engineering, Caching, and Cost Engineering is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a C++ example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small C++ project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-42"
claude "Work only on: Context Engineering, Caching, and Cost Engineering. Explain the
plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-42
claude "Work only on: Context Engineering, Caching, and Cost Engineering. Explain the
plan, then show the verification checklist."

#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Contents

- 42.1 Concepts and positioning
 - 42.1.1 Definition and use case
 - 42.1.2 Practical procedure
 - 42.1.3 Verification and boundary

- 42.2 Workflow and operating model
- 42.2.1 Definition and use case
- 42.2.2 Practical procedure
- 42.2.3 Verification and boundary
- 42.3 Common mistakes and risk control
- 42.3.1 Definition and use case
- 42.3.2 Practical procedure
- 42.3.3 Verification and boundary
- 42.4 Practice, deliverables, and review
- 42.4.1 Definition and use case
- 42.4.2 Practical procedure
- 42.4.3 Verification and boundary

42.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 42.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 42.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 42.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-42"
claude "Work only on: Context Engineering, Caching, and Cost Engineering: Concepts and
positioning. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-42
claude "Work only on: Context Engineering, Caching, and Cost Engineering: Concepts and
```

positioning. Explain the plan, then show the verification checklist."

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

42.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 42.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 42.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 42.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.

5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-42"
claude "Work only on: Context Engineering, Caching, and Cost Engineering: Workflow and
operating model. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-42
claude "Work only on: Context Engineering, Caching, and Cost Engineering: Workflow and
operating model. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

42.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 42.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 42.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 42.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-42"
claude "Work only on: Context Engineering, Caching, and Cost Engineering: Common
mistakes and risk control. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-42
claude "Work only on: Context Engineering, Caching, and Cost Engineering: Common
mistakes and risk control. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

42.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 42.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 42.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 42.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-42"
claude "Work only on: Context Engineering, Caching, and Cost Engineering: Practice,
deliverables, and review. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-42
claude "Work only on: Context Engineering, Caching, and Cost Engineering: Practice,
deliverables, and review. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format

Problem	Symptom	Fix
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed Context Engineering, Caching, and Cost Engineering as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with From Personal Capability to Team AI Coding Systems.

Chapter 43: From Personal Capability to Team AI Coding Systems

Part 9: Advanced Harness Engineering

Main case language: Python

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of From Personal Capability to Team AI Coding Systems into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what From Personal Capability to Team AI Coding Systems is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a Python example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small Python project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-43"
claude "Work only on: From Personal Capability to Team AI Coding Systems. Explain the
plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-43
claude "Work only on: From Personal Capability to Team AI Coding Systems. Explain the
plan, then show the verification checklist."

from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Contents

- 43.1 Concepts and positioning
 - 43.1.1 Definition and use case
 - 43.1.2 Practical procedure
 - 43.1.3 Verification and boundary
- 43.2 Workflow and operating model

- 43.2.1 Definition and use case
- 43.2.2 Practical procedure
- 43.2.3 Verification and boundary
- 43.3 Common mistakes and risk control
- 43.3.1 Definition and use case
- 43.3.2 Practical procedure
- 43.3.3 Verification and boundary
- 43.4 Practice, deliverables, and review
- 43.4.1 Definition and use case
- 43.4.2 Practical procedure
- 43.4.3 Verification and boundary

43.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 43.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 43.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 43.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-43"
claude "Work only on: From Personal Capability to Team AI Coding Systems: Concepts and
positioning. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-43
claude "Work only on: From Personal Capability to Team AI Coding Systems: Concepts and
positioning. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

43.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 43.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 43.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 43.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-43"
```

```
claude "Work only on: From Personal Capability to Team AI Coding Systems: Workflow and operating model. Explain the plan, then show the verification checklist."
```

```
# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-43
claude "Work only on: From Personal Capability to Team AI Coding Systems: Workflow and operating model. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

43.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 43.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 43.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 43.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.

3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-43"
claude "Work only on: From Personal Capability to Team AI Coding Systems: Common
mistakes and risk control. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-43
claude "Work only on: From Personal Capability to Team AI Coding Systems: Common
mistakes and risk control. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

43.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 43.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 43.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 43.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-43"
claude "Work only on: From Personal Capability to Team AI Coding Systems: Practice,
deliverables, and review. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-43
claude "Work only on: From Personal Capability to Team AI Coding Systems: Practice,
deliverables, and review. Explain the plan, then show the verification checklist."
```

Example

```
from pathlib import Path

project_root = Path.cwd()

def summarize_target(path: str) -> dict:
    target = project_root / path
    return {"path": str(target), "exists": target.exists(), "suffix": target.suffix}

print(summarize_target("README.md"))
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed From Personal Capability to Team AI Coding Systems as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Case Study 1: From Bug to PR in a Python Project.

Chapter 44: Case Study 1: From Bug to PR in a Python Project

Part 10: Complete Case Studies

Main case language: C#

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Case Study 1: From Bug to PR in a Python Project into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Case Study 1: From Bug to PR in a Python Project is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a C# example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small C# project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-44"
claude "Work only on: Case Study 1: From Bug to PR in a Python Project. Explain the
plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-44
claude "Work only on: Case Study 1: From Bug to PR in a Python Project. Explain the
plan, then show the verification checklist."

using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Contents

- 44.1 Concepts and positioning
- 44.1.1 Definition and use case

- 44.1.2 Practical procedure
- 44.1.3 Verification and boundary
- 44.2 Workflow and operating model
- 44.2.1 Definition and use case
- 44.2.2 Practical procedure
- 44.2.3 Verification and boundary
- 44.3 Common mistakes and risk control
- 44.3.1 Definition and use case
- 44.3.2 Practical procedure
- 44.3.3 Verification and boundary
- 44.4 Practice, deliverables, and review
- 44.4.1 Definition and use case
- 44.4.2 Practical procedure
- 44.4.3 Verification and boundary

44.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 44.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 44.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 44.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-44"
claude "Work only on: Case Study 1: From Bug to PR in a Python Project: Concepts and
```

```
positioning. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-44
claude "Work only on: Case Study 1: From Bug to PR in a Python Project: Concepts and
positioning. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

44.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 44.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 44.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 44.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-44"
claude "Work only on: Case Study 1: From Bug to PR in a Python Project: Workflow and
operating model. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-44
claude "Work only on: Case Study 1: From Bug to PR in a Python Project: Workflow and
operating model. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

44.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an

engineering behavior rather than a label.

Important subtopics:

- 44.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 44.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 44.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-44"
claude "Work only on: Case Study 1: From Bug to PR in a Python Project: Common
mistakes and risk control. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-44
claude "Work only on: Case Study 1: From Bug to PR in a Python Project: Common
mistakes and risk control. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

44.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 44.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 44.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 44.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-44"
claude "Work only on: Case Study 1: From Bug to PR in a Python Project: Practice,
deliverables, and review. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-44
claude "Work only on: Case Study 1: From Bug to PR in a Python Project: Practice,
deliverables, and review. Explain the plan, then show the verification checklist."
```

Example

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed Case Study 1: From Bug to PR in a Python Project as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

The next chapter continues with Case Study 2: Team Claude Code Toolkit.

Chapter 45: Case Study 2: Team Claude Code Toolkit

Part 10: Complete Case Studies

Main case language: C++

Commands are shown with Windows PowerShell first, followed by macOS/Linux equivalents.

Chapter Overview

This chapter turns the topic of Case Study 2: Team Claude Code Toolkit into a practical Claude Code workflow. The goal is to move from concept to executable task, then to evidence, validation, and reusable notes.

Learning Goals

- Explain what Case Study 2: Team Claude Code Toolkit is for.
- Convert the topic into a bounded Claude Code task.
- Ask Claude Code for evidence, not only fluent explanations.
- Practice with a C++ example and transfer the method to other projects.
- Finish with a checklist and a reusable artifact.

Start with a Small Case

Use a small C++ project as the observation target. The case should be small enough to run locally and concrete enough for Claude Code to inspect.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-45"
claude "Work only on: Case Study 2: Team Claude Code Toolkit. Explain the plan, then
show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-45
claude "Work only on: Case Study 2: Team Claude Code Toolkit. Explain the plan, then
show the verification checklist."

#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Contents

- 45.1 Concepts and positioning
 - 45.1.1 Definition and use case
 - 45.1.2 Practical procedure
 - 45.1.3 Verification and boundary

- 45.2 Workflow and operating model
- 45.2.1 Definition and use case
- 45.2.2 Practical procedure
- 45.2.3 Verification and boundary
- 45.3 Common mistakes and risk control
- 45.3.1 Definition and use case
- 45.3.2 Practical procedure
- 45.3.3 Verification and boundary
- 45.4 Practice, deliverables, and review
- 45.4.1 Definition and use case
- 45.4.2 Practical procedure
- 45.4.3 Verification and boundary

45.1 Concepts and positioning

Learning Goals

- Understand the role of concepts and positioning in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat concepts and positioning as an engineering behavior rather than a label.

Important subtopics:

- 45.1.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 45.1.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 45.1.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-45"
claude "Work only on: Case Study 2: Team Claude Code Toolkit: Concepts and
positioning. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-45
claude "Work only on: Case Study 2: Team Claude Code Toolkit: Concepts and
```

positioning. Explain the plan, then show the verification checklist."

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

45.2 Workflow and operating model

Learning Goals

- Understand the role of workflow and operating model in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat workflow and operating model as an engineering behavior rather than a label.

Important subtopics:

- 45.2.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 45.2.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 45.2.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.

5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-45"
claude "Work only on: Case Study 2: Team Claude Code Toolkit: Workflow and operating
model. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-45
claude "Work only on: Case Study 2: Team Claude Code Toolkit: Workflow and operating
model. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

45.3 Common mistakes and risk control

Learning Goals

- Understand the role of common mistakes and risk control in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat common mistakes and risk control as an engineering behavior rather than a label.

Important subtopics:

- 45.3.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 45.3.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 45.3.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-45"
claude "Work only on: Case Study 2: Team Claude Code Toolkit: Common mistakes and risk
control. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-45
claude "Work only on: Case Study 2: Team Claude Code Toolkit: Common mistakes and risk
control. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

45.4 Practice, deliverables, and review

Learning Goals

- Understand the role of practice, deliverables, and review in this chapter.
- Translate the idea into a prompt with goal, scope, constraints, and output format.
- Define a verification method before accepting the result.

Core Concepts

The practical value of this section is control. Claude Code works best when the task has a clear boundary, a small evidence set, and an explicit definition of done. Treat practice, deliverables, and review as an engineering behavior rather than a label.

Important subtopics:

- 45.4.1 Definition and use case: explain the problem, run a concrete observation, and record the result.
- 45.4.2 Practical procedure: explain the problem, run a concrete observation, and record the result.
- 45.4.3 Verification and boundary: explain the problem, run a concrete observation, and record the result.

Procedure

1. State the task boundary in one sentence.
2. Ask Claude Code to inspect the smallest useful set of files or documents.
3. Require file paths, commands, logs, or citations as evidence.
4. Ask for a plan before any risky action.
5. Verify the result and save reusable notes.

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-45"
claude "Work only on: Case Study 2: Team Claude Code Toolkit: Practice, deliverables,
and review. Explain the plan, then show the verification checklist."

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-45
claude "Work only on: Case Study 2: Team Claude Code Toolkit: Practice, deliverables,
and review. Explain the plan, then show the verification checklist."
```

Example

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

Exercises

- Rewrite this section as a precise Claude Code prompt.
- Ask Claude Code to produce a plan and a checklist before editing anything.
- Record one rule or prompt that can be reused in a future project.

Checklist

- [] The task has a clear goal and boundary.
- [] The output includes evidence or a verification command.
- [] The result can be repeated on a local project.
- [] The lesson has been saved as a note, rule, command, or skill.

Common Pitfalls and Troubleshooting

Problem	Symptom	Fix
The task is too broad	Claude Code reads too much or answers vaguely	Narrow the scope and request evidence
No acceptance criteria	The answer looks complete but cannot be judged	Add tests, checks, or a required output format

Problem	Symptom	Fix
Context pollution	Old rules or irrelevant files affect the answer	Ask for evidence and remove stale context
No reusable record	You repeat the same instruction later	Save the workflow as Markdown, command, or skill

Chapter Deliverables

- A chapter note.
- A reusable prompt or checklist.
- A small local case that can be repeated.

Chapter Summary

This chapter framed Case Study 2: Team Claude Code Toolkit as a repeatable Claude Code workflow. The standard is not memorization; the standard is whether you can run, verify, and reuse the workflow.

Next Chapter

This completes the handbook. Use the appendices as a working reference.

Appendices

Claude Code Command Quick Reference

Use this appendix as a compact working reference. Keep examples short, verifiable, and synchronized with the chapter workflows.

Directory and File Quick Reference

Use this appendix as a compact working reference. Keep examples short, verifiable, and synchronized with the chapter workflows.

Markdown Templates

Use this appendix as a compact working reference. Keep examples short, verifiable, and synchronized with the chapter workflows.

Prompt Template Library

Use this appendix as a compact working reference. Keep examples short, verifiable, and synchronized with the chapter workflows.

Skill Templates

Use this appendix as a compact working reference. Keep examples short, verifiable, and synchronized with the chapter workflows.

SubAgent Templates

Use this appendix as a compact working reference. Keep examples short, verifiable, and synchronized with the chapter workflows.

MCP Configuration Examples

Use this appendix as a compact working reference. Keep examples short, verifiable, and synchronized with the chapter workflows.

Hooks Examples

Use this appendix as a compact working reference. Keep examples short, verifiable, and synchronized with the chapter workflows.

Token and Cost Checklist

Use this appendix as a compact working reference. Keep examples short, verifiable, and synchronized with the chapter workflows.

Security Checklist

Use this appendix as a compact working reference. Keep examples short, verifiable, and synchronized with the chapter workflows.

Glossary

Use this appendix as a compact working reference. Keep examples short, verifiable, and synchronized with the chapter workflows.

References

- Claude Code documentation: <https://code.claude.com/docs/zh-CN/overview>
- Claude Code engineering tutorial: <https://github.com/huangjia2019/claude-code-engineering/tree/main>